

A Project Thesis on
**CAREERALIGN AI: INTELLIGENT RESUME SHORTLIST
PREDICTOR AND UPSKILLING ROADMAP GENERATOR**

Submitted in partial fulfillment of the requirements for
the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

ANUSHA YEDURADA (22021A0508)
MAMIDI GANAPATHI (22021A0542)
CHANDRABHATLA SAISANTOSH (22021A0530)
KILLO SHARON KUMARI (23025A0574)

Under the supervision of

Dr. A. KRISHNA MOHAN

Professor, Department of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING KAKINADA(A)
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA,
KAKINADA-533003, ANDHRA PRADESH, INDIA

2022-2026

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING KAKINADA
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA
KAKINADA – 533 003, AP, INDIA**



CERTIFICATE FROM THE SUPERVISOR

This is to certify that the project thesis entitled **“CAREER-ALIGN AI: INTELLIGENT RESUME SHORTLIST PREDICTOR AND UPSKILLING ROADMAP GENERATOR”** is being submitted by **ANUSHA YEDURADA (22021A0508), MAMIDI GANAPATHI (22021A0542), CHANDRABHATLA SAISANTOSH (22021A0530), KILLO SHARON KUMARI (23025A0574)** of Bachelor of Technology, Department of Computer Science and Engineering, UCEK (A), JNTUK Kakinada is a record bonafide work carried out by them under my guidance and supervision.

Signature of the Supervisor

Dr. A. Krishna Mohan

Professor

Department of CSE

UCEK (A)

JNTUK Kakinada

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING KAKINADA
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA
KAKINADA – 533 003, AP, INDIA**



CERTIFICATE FROM THE HEAD OF THE DEPARTMENT

This is to certify that the project thesis entitled **“CAREER-ALIGN AI: INTELLIGENT RESUME SHORTLIST PREDICTOR AND UPSKILLING ROADMAP GENERATOR”** is being submitted by **ANUSHA YEDURADA (22021A0508), MAMIDI GANAPATHI (22021A0542), CHANDRABHATLA SAISANTOSH (22021A0530), KILLO SHARON KUMARI (23025A0574)** of Bachelor of Technology, Department of Computer Science and Engineering, UCEK (A), JNTUK Kakinada is a record bonafide work carried out by them in the department.

Signature of Head of the Department

Dr. N. Ramakrishnaiah

Professor & HOD

Department of CSE

UCEK (A)

JNTUK Kakinada

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING KAKINADA
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA
KAKINADA – 533 003, AP, INDIA**



DECLARATION

We hereby declare that the work described in this project, entitled **“CAREERALIGN AI: INTELLIGENT RESUME SHORTLIST PREDICTOR AND UPSKILLING ROADMAP GENERATOR”**, is being submitted by us in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in the Department of Computer Science and Engineering, University College of Engineering Kakinada (A), Jawaharlal Nehru Technological University Kakinada, Kakinada – 533003, A.P., India. We further declare that this work is the result of our own investigation carried out under the supervision of **Dr. A. Krishna Mohan**, Professor, Department of Computer Science and Engineering, UCEK (A), JNTUK Kakinada. The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Place: Kakinada

Signature of the Students

Date:

Anusha Yedurada (22021A0508)

Mamidi Ganapathi (22021A0542)

Chandrabhatla Saisantosh (22021A0530)

Killo Sharon Kumari (23025A0574)

ACKNOWLEDGEMENT

We want to express our deepest gratitude to the following people for guiding us through this course and without whom this project and its results would not have been completed.

We wish to thank first our supervisor **Dr. A. Krishna Mohan, Professor, Department of CSE**, for accepting and supporting us as a project team. We sincerely convey our gratefulness and heartfelt thanks to our honorable and esteemed project guide for his supervision, guidance, encouragement, and counsel throughout our project. Without his invaluable advice and assistance, it would not have been possible for us to complete this project.

We would like to thank **Dr. N.Ramakrishnaiah**, Professor and Head of the Department of Computer Science and Engineering, UCEK, JNTUK, for his fabulous support and useful remarks throughout the project .

We are thankful to **Dr. K Padma Raju**, Principal, University College of Engineering Kakinada (Autonomous), JNTUK, for his support and enlightenment during the project.

We would like to thank our PRC members for their valuable guidelines and support during the project.

ANUSHA YEDURADA (22021A0508)

MAMIDI GANAPATHI (22021A0542)

CHANDRABHATLA SAISANTOSH (22021A0530)

KILLO SHARON KUMARI (23025A0574)

ABSTRACT

Recruitment processes today often lack transparency, as candidates do not clearly understand why their resumes are shortlisted or rejected. **CareerAlign AI** is an Artificial Intelligence-based recruitment assistance system designed to address this gap. The system analyzes three primary inputs — the candidate’s resume, job description, and predefined eligibility criteria — to predict the likelihood of a resume being shortlisted.

The proposed system integrates Natural Language Processing (NLP) and semantic matching techniques to extract structured information from unstructured resume documents. It performs eligibility validation based on constraints such as CGPA, degree requirements, and experience. Using vector embeddings and cosine similarity, the system evaluates the relevance between the resume and the job description beyond simple keyword matching. The system generates a probabilistic shortlist prediction score along with a detailed skill gap analysis highlighting missing competencies. Additionally, the system uses pre-trained Large Language Models (LLMs) to provide resume improvement suggestions and a personalized, time-bound learning roadmap to help candidates bridge identified skill gaps.

The proposed solution enhances recruitment transparency, improves candidate preparedness, and supports data-driven career development. By combining AI-driven prediction with actionable feedback, CareerAlign AI contributes to the domains of HR Technology, Education Technology, and Intelligent Decision Support Systems.

TABLE OF CONTENTS

Title	Page No.
DECLARATION	iii
ACKNOWLEDGEMENT	iii
ABSTRACT	v
TABLE OF CONTENTS	vi
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
LIST OF FIGURES	xii
1 Introduction	1
1.1 Introduction to Resume Screening Systems	1
1.2 Introduction to Artificial Intelligence in Recruitment	1
1.2.1 Working of CareerAlign AI System	2
1.3 System Overview	2
1.3.1 Semantic Similarity	3
1.3.2 Skill Matching	3
1.3.3 Eligibility Evaluation	3
1.3.4 Recommendation Module	3
1.4 Experimental Tools	3
1.5 Problem Statement	4

1.6	Research Aim and Objectives	4
1.7	Structure of Thesis	4
2	Review Of Literature	7
2.1	Literature Review	7
2.2	Comparative Analysis of Existing Systems	9
2.3	Research Gap	9
3	About the Environment	10
3.1	Google Colaboratory	10
3.2	Python Programming Language	11
3.3	Gradio Framework	12
4	System Overview	13
4.1	Existing System	13
4.1.1	Description	13
4.1.2	Architecture of Existing System	13
4.1.3	Limitations of Existing System	14
4.2	Proposed System	15
4.2.1	Description	15
4.2.2	Advantages of Proposed System	16
5	Architecture of Proposed System	17
5.1	User Interface	17
5.1.1	Interface Design	17
5.1.2	Architecture	19
5.2	Data Processing and Feature Extraction	20
5.3	Scoring and Prediction	21
5.4	Recommendation and Skill Gap Analysis	21
5.5	System Methodology	22
6	Requirement Specifications	24

6.1	Hardware Requirements	24
6.2	Software Requirements	24
6.3	System Components	27
6.3.1	Text Preprocessing Module	27
6.3.2	Skill Extraction Module	27
6.3.3	Semantic Similarity Module	28
6.3.4	Skill Matching Module	29
6.3.5	Eligibility Evaluation Module	30
6.3.6	Feature Engineering Module	31
6.3.7	Scoring and Prediction Module	33
6.3.8	Decision and Explainability Module	34
6.3.9	Recommendation Module	35
7	Workflow of the System	37
7.1	Workflow Architecture	37
7.2	System Workflow	38
8	Source Code	42
8.1	Dependency Installation	42
8.2	Imports and Configuration	42
8.3	Skill Taxonomy and Alias Dictionary	43
8.4	Loading NLP Models	44
8.5	Text Preprocessing Module	44
8.6	Skill Extraction Module	45
8.7	Semantic Similarity Module	46
8.8	Skill Matching Module	47
8.9	JD Skill Weighting Module	49
8.10	Eligibility Extraction Module	50
8.11	Eligibility Evaluation Module	51
8.12	Feature Engineering Module	52

8.13 Final Scoring and Probability Module	53
8.14 Decision and Explainability Module	54
8.15 Learning Roadmap Generation Module	55
8.16 Master ATS Pipeline	57
8.17 User Interface Implementation	59
9 Outputs	61
9.1 System Execution	61
9.2 User Interface	62
9.3 Resume Input Module	62
9.4 Job Description Input	63
9.5 Filled Input Example	64
9.6 ATS Score Output	64
9.7 Score Visualization	65
9.8 Detailed Feature Scores	66
9.9 Skill Gap Analysis	66
9.10 Analysis Explanation	67
9.11 Learning Roadmap	68
9.12 Case Analysis: Shortlisted vs Rejected	68
9.12.1 Shortlisted Case	68
9.12.2 Rejected Case	69
10 Conclusion and Future Scope	70
10.1 Conclusion	70
10.2 Future Scope	71
References	73

List of Tables

2.1 Comparison of Existing Resume Screening Systems	9
---	---

List of Abbreviations

AI	Artificial Intelligence
ATS	Applicant Tracking System
API	Application Programming Interface
DL	Deep Learning
JD	Job Description
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
PDF	Portable Document Format
TF-IDF	Term Frequency - Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator

List of Figures

3.1 Google Colaboratory Interface	11
4.1 Architecture of Existing Resume Screening System	14
5.1 Architecture of Proposed System (CareerAlign AI)	20
7.1 ATS Workflow Diagram	37
9.1 Execution of CareerAlign AI using Google Colaboratory and Gradio	61
9.2 Landing Page of CareerAlign AI	62
9.3 Resume Upload using PDF	62
9.4 Resume Input using Text	63
9.5 Job Description Input Section	63
9.6 Example of Filled Job Description and Resume Input	64
9.7 ATS Score Summary Output	64
9.8 Final Score and Shortlist Probability Visualization	65
9.9 Feature Contribution Analysis using Bar Chart	65
9.10 Detailed Feature Scores	66
9.11 Matched Skills and Missing Skills	66
9.12 Decision Explanation Output	67
9.13 AI-Generated Learning Roadmap	68
9.14 Example of Shortlisted Resume	68
9.15 Example of Rejected Resume	69

Chapter 1

Introduction

1.1 Introduction to Resume Screening Systems

Resume screening is an essential step in the recruitment process where candidate profiles are evaluated against job requirements. With the increasing number of job applicants, manual screening has become difficult, time-consuming, and prone to errors. To address this issue, organizations use Applicant Tracking Systems (ATS) to automate the filtering process. However, most existing ATS systems rely on keyword-based matching, which may fail to capture the actual meaning and relevance of candidate skills and experience. This leads to incorrect shortlisting and rejection of potential candidates. As recruitment processes continue to evolve, there is a need for more intelligent systems that can analyze resumes accurately and provide meaningful insights.

1.2 Introduction to Artificial Intelligence in Recruitment

Artificial Intelligence (AI) plays an important role in improving recruitment systems by enabling automated and intelligent decision-making. AI techniques can analyze large amounts of data and identify patterns that are not easily visible through manual methods. In recruitment, AI helps in understanding resume content, matching candidates with job roles, and improving selection accuracy. Natural Language Processing (NLP), a branch

of AI, is particularly useful for analyzing textual data such as resumes and job descriptions. It allows the system to extract key information like skills, education, and experience, making the evaluation process more efficient and reliable.

1.2.1 Working of CareerAlign AI System

The CareerAlign AI system is designed to evaluate resumes using intelligent analysis. It takes three main inputs: the candidate's resume, the job description, and predefined eligibility criteria. The system first extracts text from the resume and processes it using NLP techniques such as cleaning and tokenization. Next, the system identifies important features such as skills and qualifications. It converts both resume and job description into vector representations using embedding techniques. Cosine similarity is then calculated to measure how closely the resume matches the job description.

In addition to similarity, the system checks eligibility conditions such as CGPA, degree, and experience. Various features such as semantic similarity, skill match ratio, and keyword relevance are combined to generate a final score. A probability value is calculated to predict whether the resume will be shortlisted. The system also provides skill gap analysis and generates a personalized learning roadmap to help candidates improve.

1.3 System Overview

The system consists of multiple components that work together to evaluate resumes and predict shortlist probability. It includes semantic similarity analysis, skill matching, eligibility evaluation, feature engineering, scoring, and recommendation modules. These components ensure accurate comparison between the resume and job description and generate meaningful feedback for candidates.

1.3.1 Semantic Similarity

Semantic similarity measures how closely the meaning of two texts matches. In this system, it is used to compare the resume and job description. Instead of relying only on keywords, it captures the context and meaning of the content, resulting in more accurate matching.

1.3.2 Skill Matching

Skill matching compares the skills mentioned in the resume with those required in the job description. It considers both exact matches and partial matches using semantic similarity to determine how well a candidate fits the job role.

1.3.3 Eligibility Evaluation

Eligibility evaluation checks whether the candidate satisfies basic requirements such as degree, CGPA, and experience. Instead of strict rejection, the system applies a penalty-based approach to adjust the score, ensuring a fair evaluation.

1.3.4 Recommendation Module

The system includes a recommendation module that provides suggestions for improving the resume. It identifies missing skills and generates a personalized learning roadmap to help candidates enhance their profile.

1.4 Experimental Tools

The system is developed using Python as the primary programming language. Google Colab is used as the development environment for building and testing the backend system. Gradio is used to create a user-friendly interface for uploading resumes and job descriptions.

Natural Language Processing is performed using spaCy, and semantic similarity is computed using Sentence Transformers. Additional libraries such

as NumPy and Pandas are used for data processing and calculations. These tools together support the implementation of the system.

1.5 Problem Statement

The current recruitment process lacks transparency and depends heavily on keyword-based matching, which does not accurately reflect the true capabilities of candidates. Existing systems do not provide proper feedback, skill gap analysis, or guidance for improvement. This results in inefficient hiring processes and confusion among candidates regarding their selection or rejection.

1.6 Research Aim and Objectives

The aim of this project is to develop an intelligent resume screening system that improves accuracy and transparency in recruitment. The objectives of the project include:

- To analyze resumes and job descriptions using Natural Language Processing
- To predict shortlist probability using semantic similarity techniques
- To perform eligibility validation based on predefined criteria
- To identify skill gaps between candidate and job requirements
- To provide personalized recommendations and a learning roadmap

1.7 Structure of Thesis

This thesis is organized into ten chapters, each focusing on a specific aspect of the proposed system.

Chapter 1: Introduction

This chapter introduces the concept of resume screening systems and the

role of Artificial Intelligence in recruitment. It explains the working of the proposed CareerAlign AI system, presents the problem statement, objectives, and provides an overview of the system components.

Chapter 2: Review of Literature

This chapter discusses existing research related to resume screening systems using NLP, Machine Learning, and AI. It includes a comparative analysis of existing systems and identifies the research gap that motivates the proposed system.

Chapter 3: About the Environment

This chapter describes the tools and technologies used in the project, including Python, Google Colaboratory, and the Gradio framework, along with supporting libraries for NLP and data processing.

Chapter 4: System Overview

This chapter explains the existing resume screening systems and their limitations. It also presents the proposed system, highlighting its advantages and improvements over existing methods.

Chapter 5: Architecture of Proposed System

This chapter describes the architecture and design of the system. It explains modules such as user interface, data processing, feature extraction, scoring, and recommendation system.

Chapter 6: Requirement Specifications

This chapter outlines the hardware and software requirements needed to run the system. It also provides a detailed explanation of each module involved in the system along with formulas used for implementation.

Chapter 7: Workflow of the System

This chapter explains the step-by-step working process of the system, from input collection to final output generation including scoring, prediction,

and recommendations.

Chapter 8: Source Code

This chapter presents the implementation details of the system, including dependency setup, module-wise code explanation, and integration of different components.

Chapter 9: Outputs

This chapter shows the results of the system with screenshots, including input modules, score outputs, visualizations, skill gap analysis, and case studies (shortlisted and rejected cases).

Chapter 10: Conclusion and Future Scope

This chapter summarizes the overall work and findings of the project. It also discusses possible future enhancements and improvements to the system.

Chapter 2

Review Of Literature

2.1 Literature Review

The process of resume screening has become increasingly important due to the rapid growth in the number of job applications. Traditional recruitment methods rely on manual evaluation of resumes, which is time-consuming, inefficient, and prone to human bias. To overcome these limitations, researchers have proposed various automated systems using Artificial Intelligence (AI), Natural Language Processing (NLP), Machine Learning (ML), and Large Language Models (LLMs). [1, 2, 3, 4]

An automated resume screening system proposed in 2022 utilizes Natural Language Processing and Machine Learning techniques to filter resumes based on job requirements. The system extracts text from resumes, identifies relevant skills, and assigns scores based on predefined criteria. It generates outputs in graphical formats such as charts to assist recruiters in decision-making. This approach significantly reduces manual effort and improves efficiency in handling large volumes of resumes. However, the system primarily depends on keyword-based matching and lacks semantic understanding of the context. It also suffers from limitations such as high time complexity, lack of flexibility in handling different formats, and tightly coupled architecture, which makes it difficult to scale and modify.[5, 2]

A more advanced approach is presented in a 2026 study that focuses

on semantic resume-job matching using transformer-based models such as BERT and Sentence-BERT. This system leverages deep learning techniques to generate contextual embeddings of resumes and job descriptions, enabling it to understand semantic similarity beyond simple keyword matching. It incorporates multi-dimensional evaluation by considering skills, experience, and education alignment, and achieves high matching accuracy with improved fairness and transparency using explainable AI techniques. Despite these advancements, the system requires high computational resources and depends heavily on complex deep learning architectures. Additionally, it lacks integration of rule-based eligibility validation and does not provide clear skill gap analysis or personalized learning recommendations. [6, 3]

ProspectCV (2024) introduces a modern framework that utilizes Large Language Models (LLMs), specifically Google's Gemini Pro Vision, along with NLP techniques to evaluate resumes against job descriptions. The system generates a compatibility score, identifies missing skills, and provides personalized feedback along with recommendations for improving the resume. It also suggests learning resources to help candidates enhance their qualifications. This approach improves user experience by combining evaluation with guidance. However, the system relies heavily on LLMs, which increases computational cost and dependency on external APIs. It also lacks a structured scoring framework and does not include rule-based eligibility validation or explainable scoring mechanisms, which are important for transparency in decision-making. [7, 1, 4]

Overall, existing systems focus on specific aspects of the recruitment process such as keyword filtering, semantic matching, or feedback generation. While these approaches improve certain components individually, they do not provide a complete and integrated solution that combines semantic understanding, structured scoring, eligibility validation, skill gap analysis,

and explainable decision-making in a single framework.

2.2 Comparative Analysis of Existing Systems

Existing System	Method	Key Contribution	Limitation
Automated Resume Screener (2022)	NLP + ML	Automates resume filtering and scoring using extracted skills	Keyword-based matching, no semantic understanding, limited scalability
Semantic Resume Matching (2026)	Transformer Models (BERT, SBERT)	High accuracy semantic matching with contextual understanding and explainability	High computational cost, lacks eligibility rules and skill gap analysis
ProspectCV (2024)	LLM (Gemini) + NLP	Provides feedback, skill gap identification, and recommendations	Heavy reliance on LLMs, no structured scoring or eligibility validation

Table 2.1: Comparison of Existing Resume Screening Systems

2.3 Research Gap

From the literature review, it is observed that existing systems address only specific components of the recruitment process. Some systems focus on keyword-based filtering, while others emphasize semantic matching or feedback generation using advanced AI models. However, there is no unified system that integrates all essential components such as semantic similarity, structured scoring, eligibility validation, skill gap analysis, and personalized learning recommendations. Additionally, many systems either lack explainability or rely heavily on computationally expensive models.

Therefore, there is a need for a comprehensive and efficient system that combines rule-based and AI-based approaches to provide accurate, transparent, and scalable resume evaluation. This gap motivates the development of the proposed CareerAlign AI system.

Chapter 3

About the Environment

This chapter describes the development environment, tools, and technologies used to build the proposed system **CareerAlign AI**. The system is implemented using Python in a cloud-based environment, along with various libraries for Natural Language Processing, semantic similarity, and user interaction.

3.1 Google Colaboratory

Google Colaboratory (Colab) is a cloud-based development platform used for writing and executing Python code. It provides a Jupyter Notebook interface that allows developers to write, test, and run code directly in a web browser without requiring local installation. In this project, Google Colab is used for implementing the backend logic of the system. It allows easy integration of libraries, supports GPU acceleration when required, and provides seamless execution of machine learning and NLP-based tasks.

Some advantages of using Google Colab include:

- No need for local setup or installation
- Easy access to Python libraries
- Cloud storage integration with Google Drive
- Ability to run code from anywhere

- Interactive coding environment

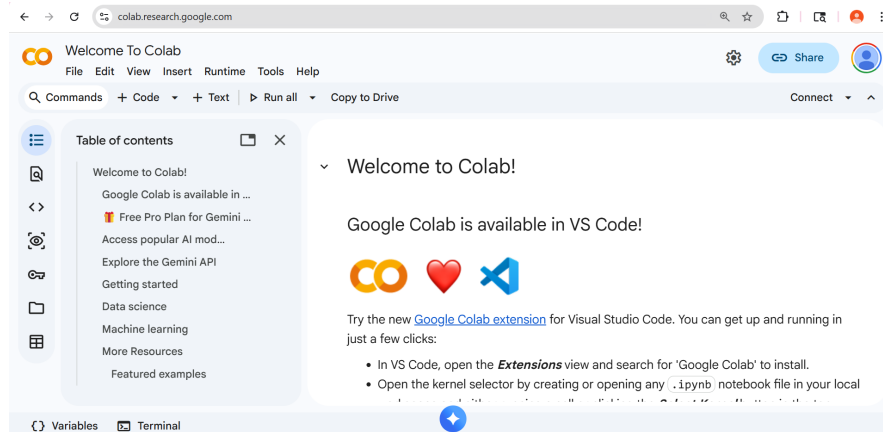


Figure 3.1: Google Colaboratory Interface

3.2 Python Programming Language

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. It is widely used in various domains such as web development, data analysis, machine learning, and artificial intelligence. In this project, Python is used as the **backend programming language** to implement all core functionalities of **CareerAlign AI**. It is responsible for tasks such as resume text processing, semantic similarity computation, eligibility evaluation, scoring, and prediction. Python provides strong support for Natural Language Processing and data handling, making it suitable for building intelligent systems.

Python offers several key features:

- **Readability:** Simple and clear syntax that makes code easy to understand and maintain.
- **Simplicity:** Requires fewer lines of code compared to other programming languages.
- **Interpreted Nature:** Executes code line by line, making debugging easier.

- **Extensive Libraries:** Provides a wide range of libraries for AI, NLP, and data processing.
- **Cross-Platform Compatibility:** Runs on multiple operating systems such as Windows, Linux, and macOS.

Python code execution is handled by an interpreter, which reads and executes the program line by line. This allows rapid development and testing, as changes can be immediately reflected without recompilation. Its flexibility and efficiency make Python an ideal choice for implementing backend systems in intelligent applications.

3.3 Gradio Framework

Gradio is used as the **frontend framework** for the proposed system **CareerAlign AI**. It provides a simple and efficient way to build interactive web interfaces for machine learning applications. In this project, Gradio is used to design a user-friendly interface that allows users to upload resumes, input job descriptions, and view the analysis results. It enables real-time interaction between the user and the backend system. Gradio eliminates the need for complex frontend technologies such as HTML, CSS, and JavaScript, making it easier to develop and deploy the system. It also ensures smooth integration with Python-based backend logic. Thus, Gradio acts as a bridge between the user and the backend processing system, ensuring smooth interaction and efficient execution of the application.

Chapter 4

System Overview

4.1 Existing System

4.1.1 Description

The existing resume screening process mainly relies on manual evaluation or basic Applicant Tracking Systems (ATS). In manual screening, recruiters review resumes individually, which is time-consuming and inefficient when dealing with a large number of applications. This process is also prone to human bias and inconsistency. Automated systems were introduced to reduce manual effort by filtering resumes based on predefined criteria. Most of these systems rely on keyword matching, where resumes are scanned for specific terms present in the job description. While this approach speeds up the process, it fails to understand the context and meaning of the content. Some advanced systems use Natural Language Processing (NLP) and machine learning techniques to improve matching accuracy. However, these systems are limited to specific functionalities such as filtering or semantic comparison. They do not provide a complete solution that includes eligibility validation, structured scoring, skill gap analysis, and personalized recommendations.

4.1.2 Architecture of Existing System

The architecture of existing resume screening systems typically follows a structured pipeline. Initially, the user submits a resume through a web por-

tal, while the job description provides the required criteria for the role. The system performs resume parsing to extract text from the document. The extracted text is then preprocessed and compared with the job description using keyword-based matching. Based on keyword overlap, the system applies rule-based scoring and threshold filtering to rank candidates. Finally, the shortlisted resumes are presented to the recruiter for further evaluation.

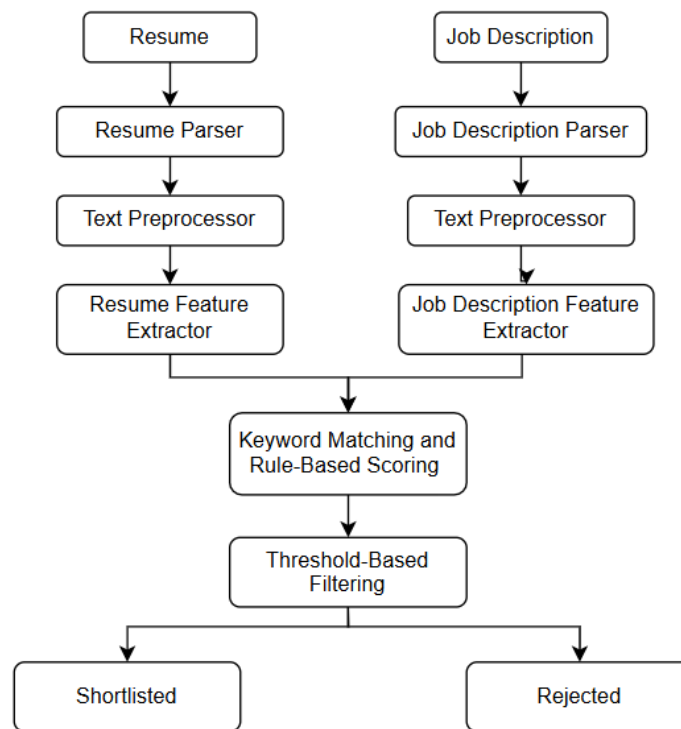


Figure 4.1: Architecture of Existing Resume Screening System

4.1.3 Limitations of Existing System

Despite their advantages, existing systems have several limitations:

1. **Keyword Dependency:** Systems rely heavily on exact keyword matching and fail to understand context.
2. **Lack of Semantic Understanding:** Cannot capture the meaning of skills expressed differently.

3. **No Eligibility Validation:** Does not consider constraints such as CGPA, degree, or experience.
4. **No Structured Scoring:** Absence of a well-defined scoring mechanism.
5. **No Skill Gap Analysis:** Does not identify missing skills for candidates.
6. **No Feedback System:** Candidates do not receive improvement suggestions.
7. **Limited Transparency:** Users are unaware of reasons for rejection or selection.

4.2 Proposed System

4.2.1 Description

The proposed system, **CareerAlign AI**, is an intelligent resume analysis system designed to improve recruitment transparency and efficiency. The system takes three inputs: resume, job description, and eligibility criteria. It processes the resume using Natural Language Processing techniques to extract important information such as skills, education, and experience.

The system uses semantic similarity techniques with embeddings and cosine similarity to compare the resume with the job description. It also performs eligibility validation based on predefined conditions such as CGPA, degree, and experience. A weighted scoring mechanism is applied to combine different factors such as semantic similarity, skill match, and keyword relevance. Based on this, the system generates a shortlist prediction score. In addition, the system performs skill gap analysis and provides personalized recommendations along with a learning roadmap to help candidates improve their profiles.

4.2.2 Advantages of Proposed System

1. **Improved Accuracy:** Uses semantic similarity instead of keyword matching.
2. **Eligibility Validation:** Considers important constraints such as degree and experience.
3. **Structured Scoring:** Provides a clear and consistent scoring mechanism.
4. **Skill Gap Analysis:** Identifies missing skills in the candidate profile.
5. **Personalized Recommendations:** Suggests improvements and a learning roadmap.
6. **Enhanced Transparency:** Clearly explains why a resume is short-listed or rejected.
7. **Efficient Processing:** Reduces manual effort and speeds up recruitment.

Chapter 5

Architecture of Proposed System

5.1 User Interface

5.1.1 Interface Design

The User Interface (UI) of the proposed system **CareerAlign AI** is developed using the Gradio framework. The primary objective of the interface is to provide a simple, intuitive, and interactive platform through which users can analyze resumes without requiring any technical knowledge. The interface acts as the entry point of the system, allowing users to upload resumes and input job descriptions. It is designed in such a way that all functionalities are accessible with minimal effort, ensuring a smooth user experience.

The UI consists of the following key components:

- **Resume Input Module:** Users can upload resumes in PDF format or directly paste resume text into the system. The interface validates the input and ensures that only valid formats are accepted.
- **Job Description Input:** Users can enter the job description in a dedicated text area. This serves as the reference for evaluating the resume against job requirements.
- **Eligibility Criteria Input:** An optional input field allows users to specify constraints such as minimum CGPA, required degree, or ex-

perience. These criteria are used for eligibility validation during analysis.

- **Analyze Resume Button:** A dedicated action button initiates the processing pipeline. Upon clicking, the system performs analysis using backend logic.
- **Processing Feedback:** The system provides visual feedback during execution, indicating that the resume analysis is in progress.
- **ATS Score Summary:** The UI displays key evaluation metrics including:
 - Semantic Similarity Score
 - Skill Match Percentage
 - Final Score
 - Shortlist Probability
 - Decision (Accepted/Rejected)
- **Visual Breakdown:** Graphical representations such as gauge charts and bar charts are used to visualize the final score, probability, and feature contributions, making the results easy to interpret.
- **Analysis Explanation:** The system provides a clear explanation of the result in simple terms, including reasons for rejection or selection and overall performance insights.
- **Skill Match Analysis:** The interface displays matched skills and identifies missing skills (skill gaps) by comparing the resume with the job description.
- **Feature Score Details:** A detailed breakdown of individual scoring components such as semantic similarity, skill match ratio, keyword density, and resume length score is presented.

- **Recommendation and Learning Roadmap:** The system generates personalized suggestions for resume improvement along with a structured learning roadmap to help candidates enhance their skills.

The interface is designed to be clean, responsive, and easy to use, ensuring that users can quickly understand the evaluation results and take necessary actions to improve their profiles.

5.1.2 Architecture

The architecture of the proposed system integrates multiple components that work together to perform resume analysis and prediction. The system takes inputs such as resume, job description, and eligibility criteria, and processes them through a series of modules. Initially, the input data is passed to the text processing module, where Natural Language Processing techniques are applied. The processed data is then sent to the feature extraction module to identify relevant attributes such as skills and qualifications. Next, the system performs semantic similarity computation using embedding models and cosine similarity. In parallel, eligibility validation is carried out based on predefined conditions. The results from these modules are combined in the scoring and prediction module, where a weighted scoring mechanism is applied. Based on the final score, the system predicts shortlist probability. Finally, the system performs skill gap analysis and generates recommendations along with a learning roadmap to guide the candidate.

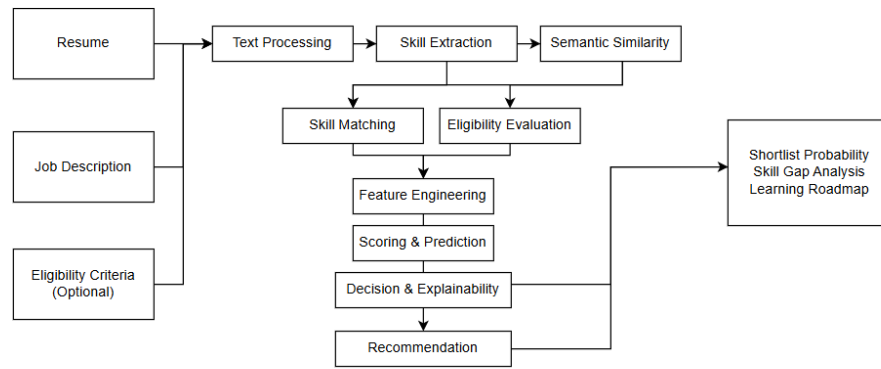


Figure 5.1: Architecture of Proposed System (CareerAlign AI)

5.2 Data Processing and Feature Extraction

The data processing stage is a critical component of the system, as it converts unstructured textual data into structured information that can be analyzed effectively. Initially, the resume and job description are provided as input to the system. The text is extracted and preprocessed using Natural Language Processing techniques. This includes operations such as removing special characters, converting text to lowercase, tokenization, and removal of stopwords. After preprocessing, the system performs feature extraction. Important attributes such as skills, educational qualifications, and experience are identified from both the resume and the job description. This step ensures that relevant information is isolated for further analysis. The system then converts the processed text into vector representations using embedding models. These embeddings capture the contextual meaning of the text rather than relying on exact keyword matches. Cosine similarity is used to compute the similarity between the resume and job description vectors. This enables the system to measure how closely the candidate profile aligns with the job requirements. This approach improves accuracy by considering the meaning of the content rather than just the presence of keywords.

5.3 Scoring and Prediction

The scoring and prediction module plays a crucial role in determining whether a resume should be shortlisted. The system considers multiple factors while calculating the final score:

- Semantic similarity score
- Skill match ratio
- Eligibility validation results
- Keyword relevance

Each of these factors is assigned a specific weight based on its importance. The weighted values are combined to generate a final score that represents the overall suitability of the candidate for the job role. A probability function is then applied to the score to estimate the likelihood of shortlisting. Based on a predefined threshold, the system classifies the resume as either shortlisted or rejected. This structured scoring mechanism ensures consistency and fairness in evaluation.

5.4 Recommendation and Skill Gap Analysis

One of the key features of the proposed system is its ability to provide intelligent recommendations to candidates. The system performs a detailed comparison between the skills present in the resume and those required in the job description. Based on this comparison, it identifies missing or weak skills.

The output includes:

- A list of missing skills
- Suggestions to improve the resume

- A personalized learning roadmap

The learning roadmap provides step-by-step guidance on how to acquire the required skills, helping candidates improve their chances of selection. This feature enhances the usability of the system by not only evaluating resumes but also guiding candidates for improvement.

5.5 System Methodology

The proposed system follows a modular and structured methodology. Each component performs a specific function, ensuring efficient processing and scalability. The workflow of the system can be summarized as follows:

1. **Input Collection:** The user provides inputs such as the resume, job description, and optional eligibility criteria. These inputs form the basis for the analysis.
2. **Text Extraction and Preprocessing:** The system extracts textual data from the resume and job description. NLP techniques such as cleaning, tokenization, and normalization are applied to prepare the data.
3. **Feature Extraction:** Relevant features such as skills, educational qualifications, and experience are identified from both the resume and job description.
4. **Text Embedding:** The processed text is converted into numerical vector representations using embedding models to capture semantic meaning.
5. **Semantic Similarity Computation:** Cosine similarity is calculated between the resume and job description embeddings to measure how closely they match.

6. **Eligibility Validation:** The system checks constraints such as minimum CGPA, degree, and experience based on user-defined criteria.
7. **Scoring and Probability Calculation:** A weighted scoring mechanism combines semantic similarity, skill match ratio, eligibility validation, and keyword relevance. Based on this score, shortlist probability is calculated.
8. **Skill Gap Analysis:** The system identifies missing or weak skills by comparing candidate skills with job requirements.
9. **Recommendation Generation:** Personalized suggestions and a structured learning roadmap are generated to help improve the candidate's profile.

This modular approach ensures that each stage of processing operates independently, allowing easy updates and enhancements without affecting the overall system. As a result, the proposed design improves accuracy, transparency, and scalability compared to traditional resume screening systems.

Chapter 6

Requirement Specifications

6.1 Hardware Requirements

The proposed system CareerAlign AI does not require high-end hardware, as most processing is efficiently handled using optimized libraries and cloud-based environments.

- **Processor:** Intel Core i5 or equivalent
- **Clock Speed:** 2 GHz or higher
- **RAM:** Minimum 8 GB (4 GB can work but 8 GB recommended for smooth execution)
- **Storage:** At least 100 GB free disk space
- **Input Devices:** Standard keyboard and mouse
- **Display:** SVGA or higher resolution monitor

The system can run on a laptop or desktop with internet access. Since development is done using Google Colab, heavy computation is not dependent on local hardware.

6.2 Software Requirements

1. Operating System

- Windows 10/11, macOS, or Linux-based systems
- 64-bit operating system recommended
- Must support Python 3.8 or higher

2. Programming Language

- **Python 3.8 or higher**
- Used for implementing the complete backend ATS pipeline including preprocessing, similarity computation, scoring, and prediction

3. Development and Execution Environment

- **Google Colaboratory (Colab)**
- Used for developing, testing, and executing the system
- Provides cloud-based execution with Jupyter Notebook interface

4. Frontend Framework

- **Gradio**
- Used to build an interactive user interface
- Allows users to upload resumes, enter job descriptions, and view outputs

5. NLP and Text Processing Libraries

- **spaCy** – Used for text preprocessing including tokenization, normalization, and lemmatization
- **en_core_web_sm model** – Pre-trained spaCy model used for efficient NLP processing
- **re (Regular Expressions)** – Used for extracting skills and patterns from resume text

- **PyMuPDF (fitz)** – Used for extracting text from PDF resumes

6. Embedding and Similarity Models

- **Sentence Transformers** – Used to convert resume and job description into vector embeddings
- **all-MiniLM-L6-v2 model** – Lightweight pre-trained transformer model used for generating embeddings
- **Cosine Similarity** – Used to measure similarity between vectors

7. Data Processing Libraries

- **NumPy** – for numerical computations
- **Pandas** – for handling structured data
- **math** – used for probability calculation (logistic function)

8. Visualization Libraries

- **Matplotlib** – Used to generate charts such as score visualization and feature breakdown

9. LLM Integration

- **Groq API / HuggingFace API** – Used for generating personalized recommendations and learning roadmap
- **LLM Models (e.g., LLaMA / Mixtral / similar transformer models)** – Used for generating human-like suggestions and roadmap content
- Rule-based fallback is used when API is not available

10. Browsers

- Google Chrome, Mozilla Firefox, or Microsoft Edge
- Used to run and interact with the Gradio interface

11. Other Requirements

- Stable internet connection
- Python virtual environment (optional)
- API keys (required for LLM integration)

6.3 System Components

The system follows a sequential pipeline where the output of each module is passed as input to the next module, ensuring a continuous flow from preprocessing to final recommendation.

6.3.1 Text Preprocessing Module

This module is responsible for extracting and preparing textual data from the resume and job description.

- Extracts text from PDF resumes using PyMuPDF
- Cleans and normalizes text (removal of special characters, lowercase conversion)
- Applies tokenization and lemmatization using spaCy
- Ensures the text is suitable for further NLP processing

6.3.2 Skill Extraction Module

This module identifies relevant skills from both the resume and job description.

- Uses regex-based matching to extract skills
- Applies skill normalization and alias mapping (e.g., “ML” → “Machine Learning”)
- Handles variations in skill representation

Formula Used:

$$S = \{ f(s) \mid s \in T \cap L \} \quad (6.1)$$

Where:

- T represents the set of tokens (words/phrases) extracted from the input text
- L represents the predefined list of known skills (skill dictionary)
- $T \cap L$ denotes the intersection of extracted tokens and known skills
- $f(s)$ represents the normalization function that maps skill variations to a standard form
- S represents the final set of extracted and normalized skills

The extracted skill set S is further compared with the job description skill set J in the skill matching module to determine the matched skills M .

6.3.3 Semantic Similarity Module

This module measures how closely the resume matches the job description.

- Converts text into vector embeddings using Sentence Transformers
- Uses the all-MiniLM-L6-v2 model
- Computes cosine similarity between vectors
- Captures contextual meaning beyond keyword matching

Formula Used:

$$S_{\text{sim}} = \frac{A \cdot B}{|A| \times |B|} \quad (6.2)$$

Where:

- A represents the embedding vector of the resume
- B represents the embedding vector of the job description

- $A \cdot B$ denotes the dot product of the two vectors
- $|A|$ and $|B|$ represent the magnitudes (lengths) of the vectors

Let S_{sim} denote the semantic similarity score obtained using cosine similarity.

6.3.4 Skill Matching Module

This module evaluates how well candidate skills match job requirements.

- Matches both exact and semantically similar skills
- Assigns weights to skills:
 - Required skills $\rightarrow$ higher weight
 - Optional skills $\rightarrow$ lower weight
- Computes overall skill match ratio

Formula Used:

$$S_{\text{skill}} = \frac{\sum_{s \in M} w_s}{\sum_{s \in J} w_s} \quad (6.3)$$

Where:

- J represents the set of all required skills from the job description
- M represents the set of matched skills between the resume and job description
- w_s represents the weight assigned to skill s
- $\sum_{s \in M} w_s$ is the total weight of matched skills
- $\sum_{s \in J} w_s$ is the total weight of all job description skills

6.3.5 Eligibility Evaluation Module

This module checks whether the candidate satisfies job constraints.

- Extracts degree, CGPA, and experience
- Applies rule-based validation
- Uses penalty-based approach instead of strict rejection

Formula Used:

$$P_{\text{elig}} = P_d \times P_c \times P_e \quad (6.4)$$

Where:

- P_d represents the penalty factor for degree mismatch
- P_c represents the penalty factor for CGPA mismatch
- P_e represents the penalty factor for experience mismatch

$$P_d = \max(0.5, 0.8^{\Delta_d}) \quad (6.5)$$

Where:

- Δ_d is the difference between required degree level and candidate degree level
- Higher Δ_d results in higher penalty
- Minimum penalty factor is limited to 0.5

$$P_c = \max(0.7, 1 - \frac{C_{\text{req}} - C_{\text{res}}}{10}) \quad (6.6)$$

Where:

- C_{req} is the required CGPA
- C_{res} is the candidate's CGPA

- The penalty increases as the CGPA shortfall increases
- Minimum penalty factor is limited to 0.7

$$P_e = \max \left(0.7, 1 - \frac{E_{req} - E_{res}}{E_{req}} \times 0.3 \right) \quad (6.7)$$

Where:

- E_{req} is the required years of experience
- E_{res} is the candidate's experience
- The penalty is proportional to experience shortfall
- Maximum reduction is limited to 30%
- Minimum penalty factor is limited to 0.7

The eligibility module applies soft penalties instead of strict rejection, ensuring that candidates with minor deficiencies are not completely eliminated but receive a reduced score.

6.3.6 Feature Engineering Module

This module generates features used for scoring.

- Semantic similarity score
- Skill match ratio
- Keyword density
- Resume length score

Formula Used (Keyword Density):

$$S_{key} = \frac{|K_{jd} \cap K_{res}|}{|K_{jd}|} \quad (6.8)$$

Where:

- K_{jd} is the set of important keywords extracted from the job description
- K_{res} is the set of words present in the resume
- $K_{jd} \cap K_{res}$ represents the keywords from the job description found in the resume
- $|K_{jd}|$ is the total number of job description keywords

Formula Used (Resume Length Score):

$$S_{len} = \begin{cases} 0.2, & \text{if } wc < 100 \\ 0.6, & \text{if } 100 \leq wc \leq 400 \\ 1.0, & \text{if } 400 < wc \leq 800 \\ 0.8, & \text{if } 800 < wc \leq 1200 \\ 0.5, & \text{if } wc > 1200 \end{cases} \quad (6.9)$$

Where:

- wc is the total number of words in the resume
- L is the normalized resume length score
- The score is highest when the resume length is within the optimal range (400–800 words)

Formula Used (Feature Normalization):

$$F_i = \frac{X_i}{100} \quad (6.10)$$

Where:

- X_i is the original feature value (e.g., similarity score in percentage)
- F_i is the normalized feature value in the range [0,1]

The computed feature scores S_{sim} , S_{skill} , S_{key} , and S_{len} , along with the eligibility multiplier P_{elig} , are combined to compute the final score S_{final} .

6.3.7 Scoring and Prediction Module

This module calculates the final score and predicts shortlist probability.

- Combines all features using weighted formula
- Applies logistic function to compute probability
- Generates final decision (Accepted / Rejected)

Formula Used (Final Score):

$$S_{\text{final}} = (0.40 \cdot S_{\text{sim}} + 0.45 \cdot S_{\text{skill}} + 0.10 \cdot S_{\text{key}} + 0.05 \cdot S_{\text{len}}) \times P_{\text{elig}} \quad (6.11)$$

Where:

- S_{sim} is the semantic similarity score between resume and job description
- S_{skill} is the weighted skill match ratio
- S_{key} is the keyword density score
- S_{len} is the resume length score
- P_{elig} is the eligibility penalty multiplier obtained from the eligibility module

All feature scores used in the final scoring equation are normalized values in the range [0,1].

Formula Used (Shortlist Probability):

$$P_{\text{shortlist}} = \frac{1}{1 + e^{-z}} \quad (6.12)$$

$$z = \frac{S_{\text{final}} - 50}{12} \quad (6.13)$$

Where:

- $P_{\text{shortlist}}$ represents the shortlist probability (range 0 to 1)
- z is the scaled score used for normalization
- S_{final} is the adjusted score after applying eligibility penalties
- e is the base of the natural logarithm

The final score is computed by combining all engineered features and applying eligibility-based penalties. This score is then transformed into a probability using a logistic function, providing an interpretable likelihood of shortlisting. The final score S_{final} is subsequently passed to the decision module for classification and explanation generation.

Justification: The logistic function is used because it maps the final score into a bounded range between 0 and 1, making it suitable for representing probabilities. It also provides smooth and continuous transitions, ensuring that small changes in score do not cause abrupt changes in prediction. By centering the function around a threshold (50 in this case), the model can clearly distinguish between likely accepted and rejected candidates. This makes the output more interpretable and aligned with real-world decision-making systems.

6.3.8 Decision and Explainability Module

This module determines the final outcome of the evaluation and provides clear, interpretable insights into the decision-making process.

- Determines final decision based on threshold
- Provides explanation for result
- Highlights weak areas and missing skills

Formula Used (Decision Rule):

$$\text{Decision} = \begin{cases} \text{Shortlist,} & \text{if } S_{\text{final}} \geq T \\ \text{Reject,} & \text{if } S_{\text{final}} < T \end{cases} \quad (6.14)$$

Where:

- S_{final} is the final score obtained from the scoring module
- T is the predefined shortlist threshold (e.g., 50)
- The decision is based purely on whether the final score exceeds the threshold

Explanation Model:

$$E = \{W, M, R\} \quad (6.15)$$

Where:

- E represents the explanation provided to the user
- W represents weak features (low scores in semantic similarity, skill match, or keyword density)
- M represents missing or weak skills identified from the job requirements
- R represents eligibility-based reasons (e.g., CGPA, degree, experience constraints)

The decision module uses the final score generated in the scoring module and applies a threshold-based rule to determine the outcome. It also generates explanations by combining weak feature analysis, missing skills, and eligibility constraints to provide transparency.

6.3.9 Recommendation Module

This module provides improvement suggestions.

- Identifies missing skills
- Generates personalized learning roadmap
- Uses LLM APIs or rule-based logic

Formula Used:

$$R = g(M, J) \quad (6.16)$$

Where:

- R represents the generated recommendations and learning roadmap
- M represents the set of missing skills identified from the skill matching module
- J represents the job description context
- $g(\cdot)$ represents the recommendation generation function, implemented using LLM APIs or rule-based logic

$$R = \bigcup_{s \in M} f(s) \quad (6.17)$$

Where:

- s represents an individual missing skill
- $f(s)$ represents recommended resources or learning steps for skill s
- R is the combined set of all recommendations

The recommendation module uses the missing skills identified in the skill matching module and generates personalized suggestions using either large language models or predefined rule-based mappings.

Chapter 7

Workflow of the System

7.1 Workflow Architecture

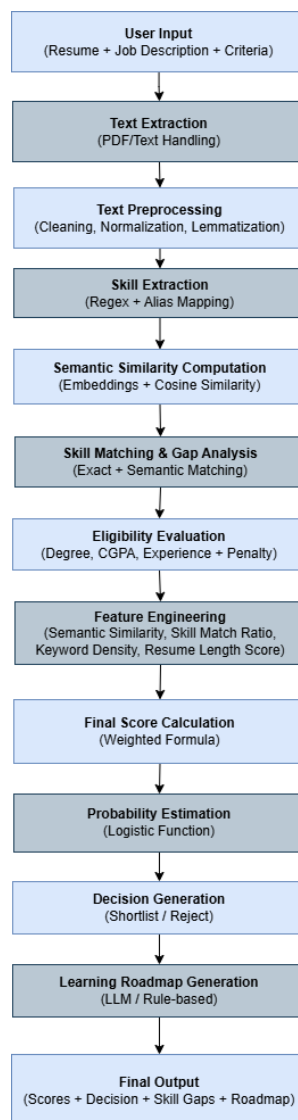


Figure 7.1: ATS Workflow Diagram

7.2 System Workflow

1. User Input and Data Submission

The workflow begins when the user interacts with the system through the Gradio-based user interface. The user provides three main inputs:

- Resume (PDF or text)
- Job Description
- Optional eligibility criteria (CGPA, degree, experience)

These inputs form the foundation for the analysis process.

2. Resume Text Extraction

Once the resume is uploaded, the system extracts textual content using the PyMuPDF library. This converts the resume document into machine-readable text.

The extracted content may contain formatting symbols or noise, which is handled in the next stage.

3. Text Preprocessing using NLP

The system processes both resume and job description using Natural Language Processing techniques.

This includes:

- Removing special characters
- Converting text to lowercase
- Tokenization (splitting into words)
- Lemmatization using spaCy

This step ensures that the text is clean, consistent, and ready for analysis.

4. Skill Extraction and Normalization

The system extracts relevant skills from both the resume and job description using regex-based matching and predefined skill lists.

- Applies skill normalization (e.g., ML → Machine Learning)
- Handles variations in skill representation

The output is a structured set of skills.

5. Semantic Similarity Computation

The processed text is converted into vector embeddings using Sentence Transformers (all-MiniLM-L6-v2 model).

- Converts text into embeddings
- Computes cosine similarity between resume and job description
- Captures contextual meaning beyond keywords

6. Skill Matching Evaluation

The system compares resume skills with job description skills.

- Identifies exact skill matches
- Identifies semantically similar skills
- Assigns weights based on importance (required vs optional)

A weighted skill match score is computed.

7. Eligibility Validation

The system checks whether the candidate satisfies job requirements such as:

- Degree
- CGPA

- Experience

A penalty-based approach is used:

- Minor mismatch → small penalty
- Major mismatch → larger penalty

This ensures fair evaluation instead of strict rejection.

8. Feature Engineering

The system generates multiple features required for scoring:

- Semantic similarity score
- Skill match score
- Keyword density
- Resume length score

All features are normalized to ensure consistent contribution.

9. Scoring and Probability Prediction

The system combines all features using a weighted formula to compute the final score.

- Applies weighted scoring
- Uses logistic function for probability

This probability represents how likely the resume is to be selected.

10. Decision and Explainability

Based on the final score, the system classifies the resume as:

- Shortlisted
- Rejected

It also provides explanations such as:

- Missing skills
- Weak areas
- Eligibility issues

This improves transparency.

11. Recommendation and Learning Roadmap

The system generates personalized suggestions based on skill gaps.

- Skills to learn
- Resume improvement tips
- Learning roadmap

These recommendations are generated using LLM APIs or rule-based logic.

12. End of Workflow

After displaying the results, the workflow is complete. The user can:

- Review the evaluation
- Understand strengths and weaknesses
- Improve their profile using recommendations

The system ensures a complete pipeline from resume submission to decision-making and improvement guidance.

Chapter 8

Source Code

8.1 Dependency Installation

```
1 # Install required libraries for the CareerAlign AI system
2
3 !pip install spacy
4 !pip install sentence-transformers
5 !pip install PyMuPDF
6 !pip install groq
7 !pip install scikit-learn
8 !pip install pandas
9 !pip install numpy
10
11 # Download spaCy English language model
12 !python -m spacy download en_core_web_sm
```

Listing 8.1: Installation of Required Libraries

8.2 Imports and Configuration

```
1
2 # Import libraries for text processing and modeling
3
4 import re                # Regular expression-based text processing
5 import os                # File handling operations
6 import math              # Mathematical computations
7 import json              # JSON data handling
8 import warnings          # Warning management
9 import numpy as np       # Numerical operations
10 import pandas as pd      # Data manipulation
11 from pathlib import Path  # File path handling
12 from typing import Optional # Optional type hints
13
14 # NLP and embedding libraries
15 import spacy
16 from sentence_transformers import SentenceTransformer, util
17
18 # Suppress unnecessary warnings
19 warnings.filterwarnings('ignore')
20
```

```

21
22 # Global Configuration Settings
23
24 CONFIG = {
25     "SHORTLIST_THRESHOLD": 70,
26     "MEDIUM_THRESHOLD": 40,
27
28     "SEMANTIC_WEIGHT": 0.50,
29     "SKILL_WEIGHT": 0.30,
30     "ELIGIBILITY_WEIGHT": 0.10,
31     "KEYWORD_WEIGHT": 0.10,
32
33     "GROQ_API_KEY": None,
34     "HF_API_KEY": None,
35
36     "USE_GPU": False
37 }

```

Listing 8.2: Importing Libraries and Global Configuration

8.3 Skill Taxonomy and Alias Dictionary

```

1
2 # Skill Taxonomy Definition
3 # List of all recognizable skills (programming, AI/ML, tools, etc.)
4
5 SKILLS_LIST = [
6     # Programming Languages
7     "python", "java", "javascript", "c++", "c", "c#", "r",
8
9     # AI / ML
10    "machine learning", "deep learning", "natural language processing",
11    "computer vision", "generative ai", "transformers", "bert", "gpt",
12
13    # Frameworks & Libraries
14    "tensorflow", "pytorch", "keras", "scikit-learn",
15
16    # Data & Tools
17    "pandas", "numpy", "sql", "mongodb", "power bi", "tableau",
18
19    # Cloud & DevOps
20    "aws", "azure", "gcp", "docker", "kubernetes", "git",
21
22    # Backend & APIs
23    "fastapi", "flask", "django",
24
25    # Frontend
26    "html", "css", "javascript", "react",
27
28    # Soft Skills
29    "communication", "teamwork", "problem solving"
30 ]
31
32
33 # Skill Alias Mapping
34 # Maps abbreviations and variations to standard skill names

```



```

35
36 SKILL_ALIASES = {
37     # Common abbreviations
38     "ml": "machine learning",
39     "dl": "deep learning",
40     "nlp": "natural language processing",
41     "cv": "computer vision",
42
43     # Programming
44     "py": "python",
45     "js": "javascript",
46
47     # Frameworks
48     "sklearn": "scikit-learn",
49     "tf": "tensorflow",
50
51     # Cloud
52     "amazon web services": "aws",
53     "google cloud": "gcp",
54     "microsoft azure": "azure",
55
56     # AI/LLM
57     "llm": "large language model",
58     "llms": "large language model",
59     "rag": "retrieval augmented generation"
60 }
61
62
63 # Convert skill list into a set for faster lookup
64 SKILLS_SET = set(SKILLS_LIST)

```

Listing 8.3: Skill Taxonomy and Alias Mapping

8.4 Loading NLP Models

```

1
2 # Load NLP and embedding models
3
4 # Load spaCy model for NLP tasks
5 nlp = spacy.load("en_core_web_sm")
6
7 # Load Sentence Transformer model for embeddings
8 embedder = SentenceTransformer("all-MiniLM-L6-v2")

```

Listing 8.4: Loading NLP and Embedding Models

8.5 Text Preprocessing Module

```

1
2 # Module 1: Text Preprocessing
3 # Handles PDF extraction and text cleaning
4
5 import fitz # PyMuPDF for PDF text extraction

```

```

6
7
8 def extract_text_from_pdf(pdf_path: str) -> str:
9     """
10    Extract text from a PDF resume.
11    """
12    try:
13        doc = fitz.open(pdf_path)
14        text = ""
15        for page in doc:
16            text += page.get_text()
17        doc.close()
18        return text.strip()
19    except Exception:
20        return ""
21
22 def preprocess_text(text: str) -> str:
23     """
24     Clean and normalize text using NLP.
25     """
26     if not text or not text.strip():
27         return ""
28
29     # Basic cleaning
30     text = text.lower()
31     text = re.sub(r"[^a-z0-9\s\+\#\.\-\/]", "", text)
32     text = re.sub(r"\s+", "", text).strip()
33
34     # NLP processing (spaCy)
35     doc = nlp(text)
36     lemmas = [
37         token.lemma_
38         for token in doc
39         if not token.is_stop and not token.is_punct and len(token.lemma_) > 1
40     ]
41
42     return " ".join(lemmas)
43
44 def safe_input(text: Optional[str], label: str = "text") -> str:
45     """
46     Validate input text.
47     """
48     if not text or not str(text).strip():
49         raise ValueError(f"Empty input received for: {label}")
50
51     return str(text).strip()

```

Listing 8.5: Module 1: Text Preprocessing

8.6 Skill Extraction Module

```

1
2 #Module 2: Skill Extraction
3 # This module identifies and normalizes skills from text using
4 # predefined skill lists and alias mappings.
5

```

```

6
7 def normalize_skill(skill: str) -> str:
8     """
9     Normalize a skill string using alias dictionary.
10
11     Parameters:
12         skill (str): Input skill
13
14     Returns:
15         str: Standardized skill name
16     """
17     skill = skill.lower().strip()
18     return SKILL_ALIASES.get(skill, skill)
19
20
21 def extract_skills(text: str) -> set:
22     """
23     Extract skills from text using:
24     1. Regex matching with predefined skill list
25     2. Alias resolution for variations
26
27     Parameters:
28         text (str): Input processed text
29
30     Returns:
31         set: Unique set of normalized skills
32     """
33     if not text:
34         return set()
35
36     text_lower = text.lower()
37     found = set()
38
39     # Match multi-word skills first (longer patterns first)
40     sorted_skills = sorted(SKILLS_LIST, key=lambda s: len(s), reverse=True)
41
42     for skill in sorted_skills:
43         pattern = r"\b" + re.escape(skill) + r"\b"
44         if re.search(pattern, text_lower):
45             found.add(normalize_skill(skill))
46
47     # Handle alias mapping (e.g., ML to Machine Learning)
48     for alias, canonical in SKILL_ALIASES.items():
49         pattern = r"\b" + re.escape(alias) + r"\b"
50         if re.search(pattern, text_lower):
51             found.add(canonical)
52
53     return found

```

Listing 8.6: Skill Extraction Module Implementation

8.7 Semantic Similarity Module

```

1
2 # Module 3: Semantic Similarity
3 # This module computes similarity between two text inputs using

```

```

4 # Sentence Transformers and cosine similarity.
5
6
7 def compute_semantic_similarity(text_a: str, text_b: str) -> float:
8     """
9     Compute cosine similarity between two texts using embeddings.
10
11     Parameters:
12         text_a (str): First text (resume)
13         text_b (str): Second text (job description)
14
15     Returns:
16         float: Similarity score in the range [0, 100]
17     """
18     if not text_a.strip() or not text_b.strip():
19         return 0.0
20
21     # Generate embeddings for both texts
22     emb_a = embedder.encode(text_a, convert_to_tensor=True)
23     emb_b = embedder.encode(text_b, convert_to_tensor=True)
24
25     # Compute cosine similarity
26     cosine_score = float(util.cos_sim(emb_a, emb_b)[0][0])
27
28     # Clamp value between 0 and 1 for safety
29     cosine_score = max(0.0, min(1.0, cosine_score))
30
31     # Convert to percentage scale
32     return round(cosine_score * 100, 2)

```

Listing 8.7: Semantic Similarity Computation

8.8 Skill Matching Module

```

1
2 # Module 4: Skill Matching
3 # Computes weighted skill matching using semantic similarity
4
5
6 # Cache to store embeddings for efficiency
7 _SKILL_EMB_CACHE: dict = {}
8
9
10 def _get_skill_embedding(skill: str):
11     """
12     Retrieve or compute embedding for a skill.
13
14     Parameters:
15         skill (str): Skill name
16
17     Returns:
18         Embedding vector
19     """
20     key = skill.lower()
21     if key not in _SKILL_EMB_CACHE:
22         _SKILL_EMB_CACHE[key] = embedder.encode(skill, convert_to_tensor=True)

```

```

23     return _SKILL_EMB.CACHE[key]
24
25
26 def compute_skill_match(
27     resume_skills: set,
28     jd_skills: set,
29     jd_skill_weights: dict = None,
30     semantic_threshold: float = 0.65
31 ) -> dict:
32     """
33     Compute weighted skill matching using semantic similarity.
34
35     Returns:
36         Dictionary containing matched skills, missing skills,
37         and skill match scores.
38     """
39
40     if not jd_skills:
41         return {
42             "matched_skills": set(),
43             "missing_skills": set(),
44             "skill_match_ratio": 0.0,
45             "skill_match_score": 0.0,
46             "match_details": [],
47         }
48
49     if jd_skill_weights is None:
50         jd_skill_weights = {s: 1.0 for s in jd_skills}
51
52     # Compute embeddings
53     resume_embeddings = {s: _get_skill_embedding(s) for s in resume_skills}
54     jd_embeddings = {s: _get_skill_embedding(s) for s in jd_skills}
55
56     matched = set()
57     match_details = []
58
59     for jd_skill in jd_skills:
60         best_match = None
61         best_sim = 0.0
62         jd_emb = jd_embeddings[jd_skill]
63
64         for res_skill, res_emb in resume_embeddings.items():
65             sim = float(util.cos_sim(jd_emb, res_emb)[0][0])
66
67             if sim > best_sim:
68                 best_sim = sim
69                 best_match = res_skill
70
71         if best_sim >= semantic_threshold:
72             matched.add(jd_skill)
73             match_details.append((jd_skill, best_match, round(best_sim, 3)))
74         else:
75             match_details.append((jd_skill, None, round(best_sim, 3)))
76
77     missing = jd_skills - matched
78
79     total_weight = sum(jd_skill_weights.values())
80     matched_weight = sum(jd_skill_weights[s] for s in matched)

```

```

81
82 weighted_ratio = matched_weight / total_weight if total_weight > 0 else 0.0
83
84 return {
85     "matched_skills": matched,
86     "missing_skills": missing,
87     "skill_match_ratio": round(weighted_ratio, 4),
88     "skill_match_score": round(weighted_ratio * 100, 2),
89     "match_details": match_details,
90 }

```

Listing 8.8: Skill Matching Module Implementation

8.9 JD Skill Weighting Module

```

1
2 def extract_jd_skill_weights(jd_text: str, jd_skills: set) -> dict:
3     """
4     Assign weights to job description skills based on context.
5
6     – Required skills – weight 1.0
7     – Optional skills – weight 0.7
8     – Others – weight 0.85
9     """
10
11     if not jd_skills:
12         return {}
13
14     jd_text_lower = jd_text.lower()
15
16     required_text = jd_text_lower
17     optional_text = ""
18
19     # Identify optional section
20     patterns = [
21         r'(?:(nice\s*to\s*have|preferred|plus|bonus)',
22         r'(?:(optional|desired|advantageous)',
23     ]
24
25     for pat in patterns:
26         match = re.search(r'\b' + pat + r'\b', jd_text_lower)
27         if match:
28             required_text = jd_text_lower[:match.start()]
29             optional_text = jd_text_lower[match.start():]
30             break
31
32     weights = {}
33
34     for skill in jd_skills:
35         in_required = skill.lower() in required_text
36         in_optional = skill.lower() in optional_text
37
38         if in_required and not in_optional:
39             weights[skill] = 1.0
40         elif in_optional and not in_required:
41             weights[skill] = 0.7

```

```

42     else:
43         weights[skill] = 0.85
44
45     return weights

```

Listing 8.9: Job Description Skill Weighting

8.10 Eligibility Extraction Module

```

1
2 # Module 5: Eligibility Extraction and Evaluation Module
3 # Degree Patterns and Levels
4
5 DEGREE_PATTERNS = {
6     "phd": ["ph.d", "phd", "doctorate"],
7     "masters": ["m.tech", "m.e.", "msc", "ms"],
8     "bachelors": ["b.tech", "b.e.", "bsc", "bs"],
9     "diploma": ["diploma", "polytechnic"],
10 }
11
12 DEGREE_LEVEL = {"phd": 4, "masters": 3, "bachelors": 2, "diploma": 1, "none": 0}
13
14
15 def extract_degree(text: str) -> str:
16     """Extract highest degree from text."""
17     text_lower = text.lower()
18     for degree, patterns in DEGREE_PATTERNS.items():
19         for p in patterns:
20             if p in text_lower:
21                 return degree
22     return "none"
23
24
25 def extract_cgpa(text: str) -> Optional[float]:
26     """Extract CGPA using regex."""
27     patterns = [
28         r"cgpa[\.:s]*(\d{0-9}\.\d{0-9}){1,2})",
29         r"gpa[\.:s]*(\d{0-9}\.\d{0-9}){1,2})",
30         r"(\d{0-9}\.\d{0-9}){1,2})\s*/\s*10",
31     ]
32     for pattern in patterns:
33         match = re.search(pattern, text.lower())
34         if match:
35             try:
36                 val = float(match.group(1))
37                 return val if val <= 10 else None
38             except ValueError:
39                 continue
40     return None
41
42
43 def extract_experience_years(text: str) -> float:
44     """Extract years of experience."""
45     pattern = r"(\d+\.?\d*)\s*years?"
46     matches = re.findall(pattern, text.lower())
47     values = [float(m) for m in matches if float(m) <= 60]

```

```
48 return max(values) if values else 0.0
```

Listing 8.10: Eligibility Information Extraction

8.11 Eligibility Evaluation Module

```
1
2 def check_eligibility(
3     resume_text: str,
4     jd_text: str,
5     min_degree: Optional[str] = None,
6     min_cgpa: Optional[float] = None,
7     min_experience: Optional[float] = None,
8 ) -> dict:
9     """
10    Evaluate eligibility and compute penalty multiplier.
11    """
12
13    resume_degree = extract_degree(resume_text)
14    resume_cgpa = extract_cgpa(resume_text)
15    resume_exp = extract_experience_years(resume_text)
16
17    if min_degree is None:
18        min_degree = extract_degree(jd_text) or 'none'
19    if min_cgpa is None:
20        min_cgpa = extract_cgpa(jd_text)
21    if min_experience is None:
22        min_experience = extract_experience_years(jd_text) or 0.0
23
24    eligible = True
25    multiplier = 1.0
26    reasons = []
27
28    # Degree check
29    if DEGREE_LEVEL.get(resume_degree, 0) < DEGREE_LEVEL.get(min_degree, 0):
30        multiplier *= 0.8
31        eligible = False
32        reasons.append("Degree requirement not met")
33
34    # CGPA check
35    if min_cgpa and (resume_cgpa is None or resume_cgpa < min_cgpa):
36        multiplier *= 0.9
37        eligible = False
38        reasons.append("CGPA below requirement")
39
40    # Experience check
41    if resume_exp < min_experience:
42        multiplier *= 0.85
43        eligible = False
44        reasons.append("Experience below requirement")
45
46    return {
47        "eligible": eligible,
48        "penalty_multiplier": round(multiplier, 4),
49        "resume_degree": resume_degree,
50        "resume_cgpa": resume_cgpa,
```



```

51     "resume_experience": resume_exp,
52     "eligibility_reasons": reasons,
53 }

```

Listing 8.11: Eligibility Evaluation and Penalty Calculation

8.12 Feature Engineering Module

```

1
2 # Module 6: Feature Engineering
3 # This module computes additional features used in scoring.
4
5
6 def compute_keyword_density(resume_text: str, jd_text: str) -> float:
7     """
8     Compute keyword density:
9     Fraction of unique job description keywords found in resume.
10
11     Returns:
12         float: value in range [0, 1]
13     """
14     if not jd_text.strip() or not resume_text.strip():
15         return 0.0
16
17     # Extract keywords from job description
18     jd_doc = nlp(jd_text.lower())
19     jd_keywords = set(
20         token.lemma_
21         for token in jd_doc
22         if not token.is_stop and not token.is_punct and len(token.lemma_) > 2
23     )
24
25     if not jd_keywords:
26         return 0.0
27
28     # Compare with resume words
29     resume_words = set(resume_text.lower().split())
30     hits = sum(1 for kw in jd_keywords if kw in resume_words)
31
32     return round(hits / len(jd_keywords), 4)
33
34
35 def compute_resume_length_score(resume_text: str) -> float:
36     """
37     Score resume length based on word count.
38
39     Optimal range: 400–800 words
40     Returns:
41         float: score in range [0, 1]
42     """
43     word_count = len(resume_text.split())
44
45     if word_count < 100:
46         return 0.2
47     elif word_count <= 400:
48         return 0.6

```

```

49     elif word_count <= 800:
50         return 1.0
51     elif word_count <= 1200:
52         return 0.8
53     else:
54         return 0.5
55
56
57 def engineer_features(
58     semantic_similarity: float,
59     weighted_skill_match_ratio: float,
60     keyword_density: float,
61     resume_length_score: float,
62 ) -> dict:
63     """
64     Combine all features into normalized values.
65
66     Returns:
67         dict: feature values scaled to [0,1]
68     """
69     return {
70         "semantic_similarity": round(semantic_similarity / 100, 4),
71         "weighted_skill_match_ratio": round(weighted_skill_match_ratio, 4),
72         "keyword_density": round(keyword_density, 4),
73         "resume_length_score": round(resume_length_score, 4),
74     }

```

Listing 8.12: Feature Engineering Module Implementation

8.13 Final Scoring and Probability Module

```

1
2 # Modules 7 & 8: Final Scoring and Probability
3 # This module combines engineered features and computes the final score,
4 # followed by probability estimation using a logistic function.
5
6
7 def compute_final_score(features: dict, eligibility_penalty: float = 1.0) -> dict:
8     """
9     Compute final score and shortlist probability.
10
11     Weights:
12     - Semantic Similarity: 0.40
13     - Skill Match: 0.45
14     - Keyword Density: 0.10
15     - Resume Length: 0.05
16     """
17
18     # Weighted feature combination
19     raw_score = (
20         0.40 * features.get("semantic_similarity", 0.0)
21         + 0.45 * features.get("weighted_skill_match_ratio", 0.0)
22         + 0.10 * features.get("keyword_density", 0.0)
23         + 0.05 * features.get("resume_length_score", 0.0)
24     )
25

```

```

26 # Apply eligibility penalty
27 adjusted_score = raw_score * eligibility_penalty
28
29 # Convert to percentage scale
30 final_score = round(adjusted_score * 100, 2)
31
32 # Logistic function for probability estimation
33 scaled = (final_score - 50) / 12
34 probability = round(1 / (1 + exp(-scaled)), 4)
35
36 return {
37     "final_score": final_score,
38     "shortlist_probability": probability,
39     "raw_score": round(raw_score, 4),
40     "eligibility_penalty": eligibility_penalty,
41 }

```

Listing 8.13: Final Scoring and Probability Computation

8.14 Decision and Explainability Module

```

1
2 # Module 9: Decision and Explainability
3 # This module generates the final decision and explanation.
4
5
6 def make_decision(
7     final_score: float,
8     features: dict,
9     missing_skills: set,
10    jd_skill_weights: dict = None,
11    eligibility_reasons: list = None,
12    eligibility_penalty: float = 1.0,
13 ) -> dict:
14     """
15     Generate decision and explanation based on final score and feature analysis.
16     """
17
18     # Identify weak feature areas
19     weak_features = []
20
21     if features["semantic_similarity"] < 0.4:
22         weak_features.append("Low semantic alignment with job description")
23
24     if features["weighted_skill_match_ratio"] < 0.4:
25         weak_features.append("Insufficient core skill coverage")
26
27     if features["keyword_density"] < 0.3:
28         weak_features.append("Low keyword density")
29
30     if eligibility_penalty < 0.8:
31         weak_features.append(f"Eligibility penalty applied ({eligibility_penalty:.2f})")
32
33     # Identify important missing skills
34     core_missing = set()
35

```

```

36 if jd_skill_weights:
37     core_missing = {
38         s for s in missing_skills
39         if jd_skill_weights.get(s, 1.0) > 0.8
40     }
41
42 if not core_missing:
43     core_missing = missing_skills
44
45 # Final decision based on threshold
46 if final_score > CONFIG["SHORTLIST_THRESHOLD"]:
47     decision = "Shortlist"
48     primary_reason = f"Score {final_score:.1f} exceeds threshold"
49 else:
50     decision = "Reject"
51     primary_reason = f"Score {final_score:.1f} below threshold"
52
53 # Build explanation
54 explanation_parts = [primary_reason]
55
56 if weak_features:
57     explanation_parts.append("Weak areas: " + ", ".join(weak_features))
58
59 if eligibility_reasons:
60     explanation_parts.append("Eligibility issues: " + ", ".join(eligibility_reasons))
61
62 if core_missing:
63     top_missing = sorted(core_missing)[:8]
64     explanation_parts.append("Missing core skills: " + ", ".join(top_missing))
65
66 return {
67     "decision": decision,
68     "weak_features": weak_features,
69     "explanation": " | ".join(explanation_parts),
70 }

```

Listing 8.14: Decision and Explainability Module

8.15 Learning Roadmap Generation Module

```

1
2 # Module 10: LLM Roadmap Generator
3 # Generates learning roadmap using LLM (Groq / HuggingFace) or fallback.
4
5
6 def _groq_llm(prompt: str, api_key: str) -> str:
7     """Generate roadmap using Groq API."""
8     try:
9         from groq import Groq
10        client = Groq(api_key=api_key)
11
12        response = client.chat.completions.create(
13            model="llama-3.3-70b-versatile",
14            messages=[{"role": "user", "content": prompt}],
15            max_tokens=1024,
16            temperature=0.7,

```

```

17     )
18
19     return response.choices[0].message.content.strip()
20
21 except Exception as e:
22     raise RuntimeError(f"Groq API failed: {e}")
23
24
25 def _hf_llm(prompt: str, api_key: str) -> str:
26     """Generate roadmap using HuggingFace API."""
27     import requests
28
29     headers = {"Authorization": f"Bearer {api_key}"}
30     payload = {
31         "inputs": prompt,
32         "parameters": {"max_new_tokens": 512, "temperature": 0.7},
33     }
34
35     try:
36         r = requests.post(
37             "https://api-inference.huggingface.co/models/mistralai/Mistral-7B-Instruct-v0.2",
38             headers=headers,
39             json=payload,
40             timeout=30,
41         )
42
43         result = r.json()
44
45         if isinstance(result, list):
46             return result[0].get("generated_text", "").strip()
47
48         return str(result)
49
50 except Exception as e:
51     raise RuntimeError(f"HuggingFace API failed: {e}")
52
53
54 def _rule_based_roadmap(missing_skills: set, jd_skills: set) -> str:
55     """Fallback roadmap generation without LLM."""
56
57     lines = ["Rule-Based Learning Roadmap", "=" * 40]
58
59     if not missing_skills:
60         lines.append("Candidate already matches job requirements.")
61     else:
62         lines.append("Skills to Improve:\n")
63
64         for i, skill in enumerate(sorted(missing_skills), 1):
65             lines.append(f"{i}. {skill.title()}")
66             lines.append("    -> Learn via courses, documentation, and projects")
67
68     lines.append("\nSuggested Timeline: 3-6 months")
69     return "\n".join(lines)
70
71
72 def generate_roadmap(
73     missing_skills: set,
74     jd_skills: set,

```

```

75     resume_text: str,
76     job_description: str,
77 ) -> str:
78     """
79     Generate personalized roadmap using LLM or fallback method.
80     """
81
82     if not missing_skills:
83         return "Candidate matches all required skills."
84
85     prompt = f"""
86     You are a career advisor.
87
88     Job Description:
89     {job_description[:400]}
90
91     Missing Skills:
92     {' , '.join(sorted(missing_skills))}
93
94     Provide:
95     1. Resume improvement tips
96     2. 3-month learning roadmap
97     3. Recommended resources
98     """
99
100    # Try Groq API
101    if CONFIG.get("GROQ_API_KEY"):
102        try:
103            return _groq_llm(prompt, CONFIG["GROQ_API_KEY"])
104        except:
105            pass
106
107    # Try HuggingFace API
108    if CONFIG.get("HF_API_KEY"):
109        try:
110            return _hf_llm(prompt, CONFIG["HF_API_KEY"])
111        except:
112            pass
113
114    # Fallback
115    return _rule_based_roadmap(missing_skills, jd_skills)

```

Listing 8.15: LLM-Based Roadmap Generation Module

8.16 Master ATS Pipeline

```

1
2 # Master ATS Pipeline
3 # Integrates all modules into a single workflow.
4
5
6 def run_ats_pipeline(
7     resume_input: str,
8     job_description: str,
9     eligibility_criteria: Optional[dict] = None,
10    is_pdf: bool = False,

```

```

11 ) -> dict:
12     """
13     End-to-end pipeline for resume evaluation.
14     """
15
16     # Step 1: Input Handling
17     if is_pdf:
18         raw_resume = extract_text_from_pdf(resume_input)
19         raw_resume = safe_input(raw_resume, "resume PDF")
20     else:
21         raw_resume = safe_input(resume_input, "resume text")
22
23     raw_jd = safe_input(job_description, "job description")
24
25     # Step 2: Preprocessing
26     clean_resume = preprocess_text(raw_resume)
27     clean_jd = preprocess_text(raw_jd)
28
29     # Step 3: Skill Extraction
30     resume_skills = extract_skills(raw_resume)
31     jd_skills = extract_skills(raw_jd)
32     jd_skill_weights = extract_jd_skill_weights(raw_jd, jd_skills)
33
34     # Step 4: Semantic Similarity
35     similarity_score = compute_semantic_similarity(clean_resume, clean_jd)
36
37     # Step 5: Skill Matching
38     skill_result = compute_skill_match(resume_skills, jd_skills, jd_skill_weights)
39
40     # Step 6: Eligibility Evaluation
41     criteria = eligibility_criteria or {}
42     elig_result = check_eligibility(
43         raw_resume,
44         raw_jd,
45         min_degree=criteria.get("min_degree"),
46         min_cgpa=criteria.get("min_cgpa"),
47         min_experience=criteria.get("min_experience"),
48     )
49
50     # Step 7: Feature Engineering
51     keyword_density = compute_keyword_density(clean_resume, clean_jd)
52     resume_length_score = compute_resume_length_score(raw_resume)
53
54     features = engineer_features(
55         semantic_similarity=similarity_score,
56         weighted_skill_match_ratio=skill_result["skill_match_ratio"],
57         keyword_density=keyword_density,
58         resume_length_score=resume_length_score,
59     )
60
61     # Step 8: Final Scoring
62     score_result = compute_final_score(
63         features,
64         elig_result["penalty_multiplier"]
65     )
66
67     # Step 9: Decision Making
68     decision_result = make_decision(

```

```

69     final_score=score_result["final_score"],
70     features=features,
71     missing_skills=skill_result["missing_skills"],
72     jd_skill_weights=jd_skill_weights,
73     eligibility_reasons=elig_result["eligibility_reasons"],
74     eligibility_penalty=elig_result["penalty_multiplier"],
75 )
76
77 # Step 10: Learning Roadmap
78 roadmap = generate_roadmap(
79     missing_skills=skill_result["missing_skills"],
80     jd_skills=jd_skills,
81     resume_text=raw_resume,
82     job_description=raw_jd,
83 )
84
85 return {
86     "similarity_score": similarity_score,
87     "skill_match_score": skill_result["skill_match_score"],
88     "missing_skills": list(skill_result["missing_skills"]),
89     "eligibility": elig_result["eligible"],
90     "final_score": score_result["final_score"],
91     "shortlist_probability": score_result["shortlist_probability"],
92     "decision": decision_result["decision"],
93     "explanation": decision_result["explanation"],
94     "learning_roadmap": roadmap,
95 }

```

Listing 8.16: End-to-End ATS Pipeline

8.17 User Interface Implementation

```

1
2 import gradio as gr
3 from ats_pipeline import run_ats_pipeline
4
5 # Wrapper function to connect UI with backend pipeline
6 def analyze(resume_text, job_description, eligibility_criteria):
7
8     # Call backend pipeline
9     result = run_ats_pipeline(
10         resume_input=resume_text,
11         job_description=job_description,
12         eligibility_criteria=eligibility_criteria
13     )
14
15     return (
16         result.get("similarity_score", 0),
17         result.get("skill_match_score", 0),
18         result.get("final_score", 0),
19         result.get("shortlist_probability", 0),
20         result.get("decision", ""),
21         ", ".join(result.get("missing_skills", [])),
22         result.get("learning_roadmap", "")
23     )
24

```



```

25
26 # Build Gradio Interface
27 interface = gr.Interface(
28     fn=analyze,
29     inputs=[
30         gr.Textbox(label="Resume Text"),
31         gr.Textbox(label="Job Description"),
32         gr.Textbox(label="Eligibility Criteria (Optional)")
33     ],
34     outputs=[
35         gr.Number(label="Similarity Score"),
36         gr.Number(label="Skill Match Score"),
37         gr.Number(label="Final Score"),
38         gr.Number(label="Shortlist Probability"),
39         gr.Textbox(label="Decision"),
40         gr.Textbox(label="Missing Skills"),
41         gr.Textbox(label="Learning Roadmap")
42     ],
43     title="CareerAlign AI",
44     description="AI-based Resume Analysis and Recommendation System"
45 )
46
47 interface.launch()

```

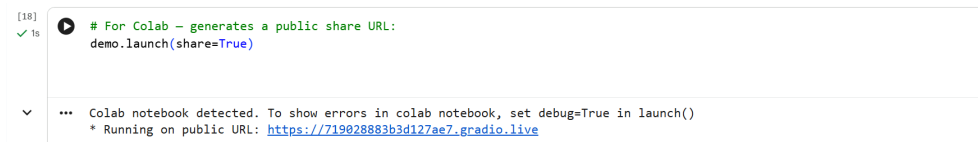
Listing 8.17: Gradio-Based User Interface

Chapter 9

Outputs

This chapter presents the outputs generated by the proposed system **CareerAlign AI**. The outputs demonstrate the working of the system, including system execution, user interaction, resume analysis, scoring, visualization, skill gap identification, and recommendation generation.

9.1 System Execution



```
[18] ✓ 1s • # For Colab - generates a public share URL:
      demo.launch(share=True)

  ... Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
  * Running on public URL: https://719028883b3d127ae7.gradio.live
```

Figure 9.1: Execution of CareerAlign AI using Google Colaboratory and Gradio

The system is executed using Google Colaboratory. The application is launched using the Gradio framework, which generates a public URL. This URL allows users to access the web-based interface and interact with the system for resume analysis without requiring local installation.

9.2 User Interface

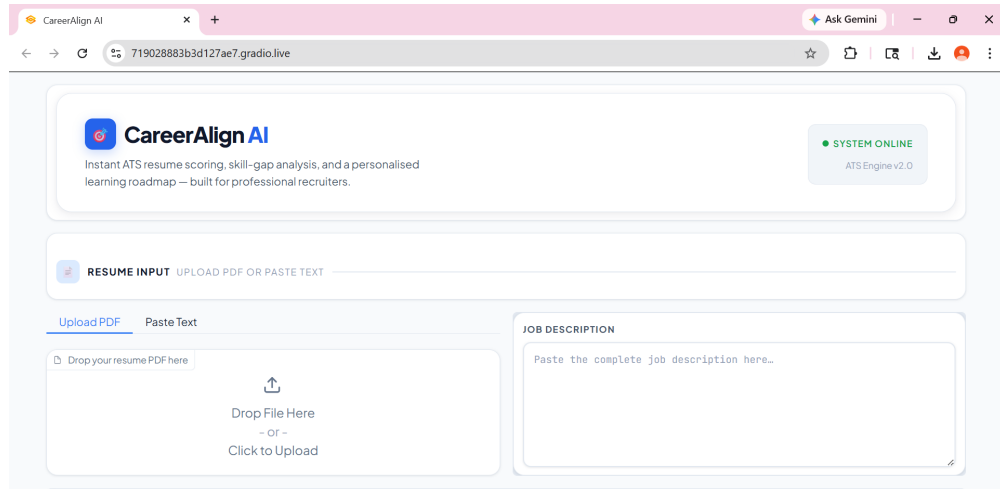


Figure 9.2: Landing Page of CareerAlign AI

The landing page provides a clean and interactive interface where users can upload resumes, enter job descriptions, and provide eligibility criteria. It ensures ease of use and smooth interaction with the system.

9.3 Resume Input Module

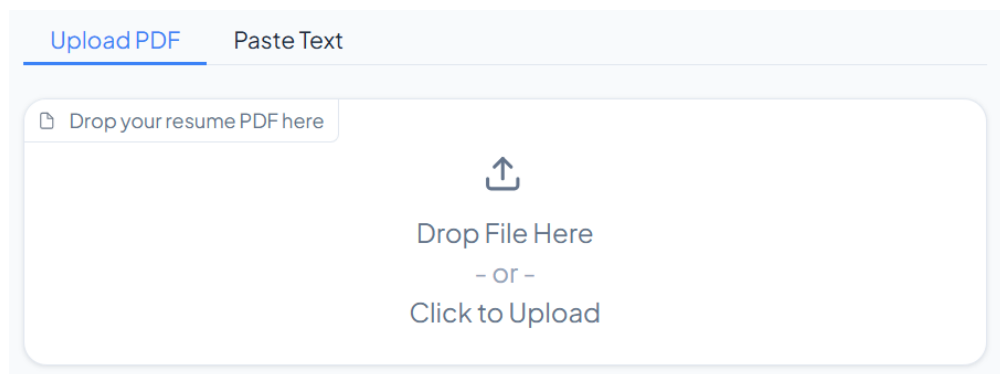


Figure 9.3: Resume Upload using PDF

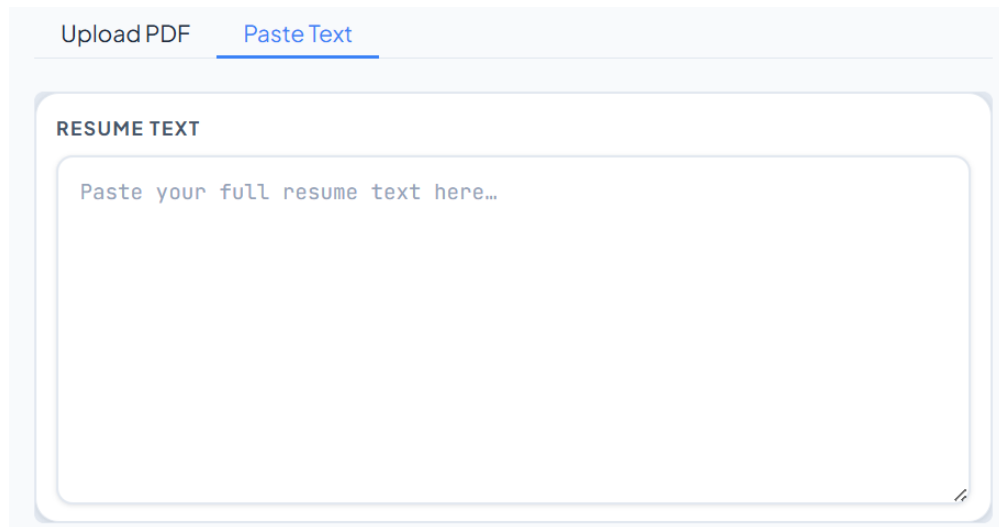
The image shows a web interface for resume input. At the top, there are two tabs: 'Upload PDF' and 'Paste Text'. The 'Paste Text' tab is selected and highlighted with a blue underline. Below the tabs is a large, rounded rectangular text input area. The title 'RESUME TEXT' is positioned at the top left of this area. Inside the input area, there is a placeholder text that reads 'Paste your full resume text here...'. The entire form is set against a light blue background.

Figure 9.4: Resume Input using Text

The system allows users to input resumes in two ways: uploading a PDF file or directly pasting resume text. This flexibility ensures that users can choose a convenient method while enabling the system to process the resume effectively.

9.4 Job Description Input

The image shows a web interface for job description input. It features a single tab labeled 'JOB DESCRIPTION' at the top left. Below the tab is a large, rounded rectangular text input area. Inside the input area, there is a placeholder text that reads 'Paste the complete job description here...'. The entire form is set against a light blue background.

Figure 9.5: Job Description Input Section

The job description is provided by the user and acts as the reference for evaluating the resume. It includes details such as required skills, responsibilities, and qualifications.

9.5 Filled Input Example

The screenshot shows a web interface for resume analysis. At the top, there's a 'RESUME INPUT' section with a sub-header 'UPLOAD PDF OR PASTE TEXT'. Below this, there are two tabs: 'Upload PDF' and 'Paste Text'. The 'Paste Text' tab is active, showing a text area with the following resume content:

RESUME TEXT

Objective:
Computer Science student with strong foundation in Machine Learning, NLP, and Data Analysis.

Education:
B.Tech in Computer Science Engineering
University College of Engineering, Kakinada
CGPA: 8.6

Skills:
Python, Machine Learning, NLP, Data Analysis, SQL, Pandas, NumPy, Scikit-Learn, FastAPI, Git

Experience:
Machine Learning Intern - Next2Technologies (6 months)
- Worked on NLP tasks such as text classification
- Performed data preprocessing and feature engineering
- Built APIs using FastAPI

To the right of the resume input is a 'JOB DESCRIPTION' section. It contains the following text:

JOB DESCRIPTION

Job Title: Machine Learning Engineer Intern

Responsibilities:

- Develop machine learning and deep learning models
- Work on NLP tasks such as text classification and semantic similarity
- Perform data preprocessing and feature engineering
- Build APIs for deployment using FastAPI
- Work with cloud platforms and container tools

Required Skills:
Python, Machine Learning, Deep Learning, NLP, Data Analysis, SQL

Preferred Skills:
TensorFlow, PyTorch, AWS, Docker

Eligibility Criteria:

- Minimum CGPA: 7.0
- 0-1 years experience

Figure 9.6: Example of Filled Job Description and Resume Input

This figure shows a sample input where both the resume and job description are provided. These inputs are used by the system to perform analysis and generate results.

9.6 ATS Score Output

The screenshot shows the 'ATS SCORE SUMMARY' section, which displays key metrics from the analysis. The table below represents the data shown in the image:

Similarity	Skill Match	Final Score	Probability	Decision
76.06	79.66	59.75	0.6926	✖ Reject

Figure 9.7: ATS Score Summary Output

The system generates key metrics including semantic similarity, skill match score, final score, shortlist probability, and decision (Shortlisted or Rejected). These outputs provide a clear evaluation of the resume.

9.7 Score Visualization

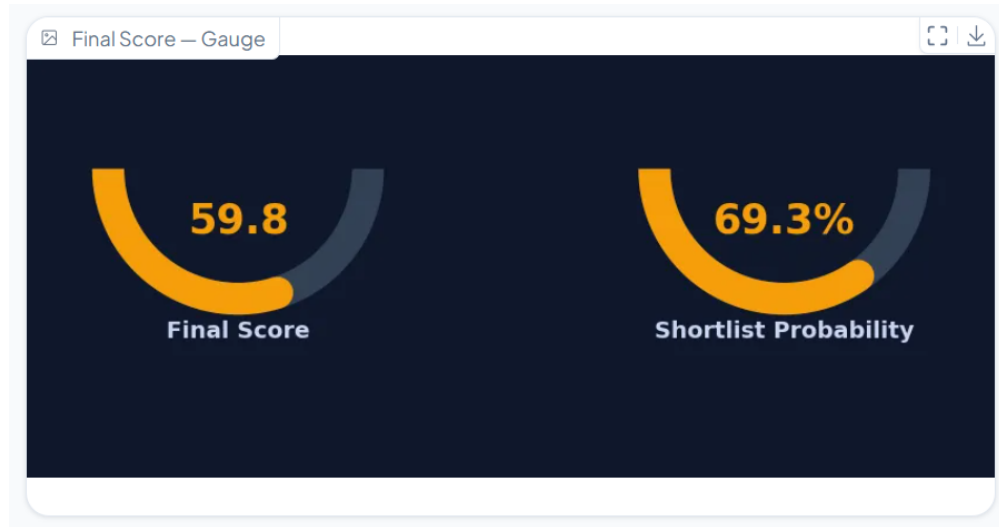


Figure 9.8: Final Score and Shortlist Probability Visualization

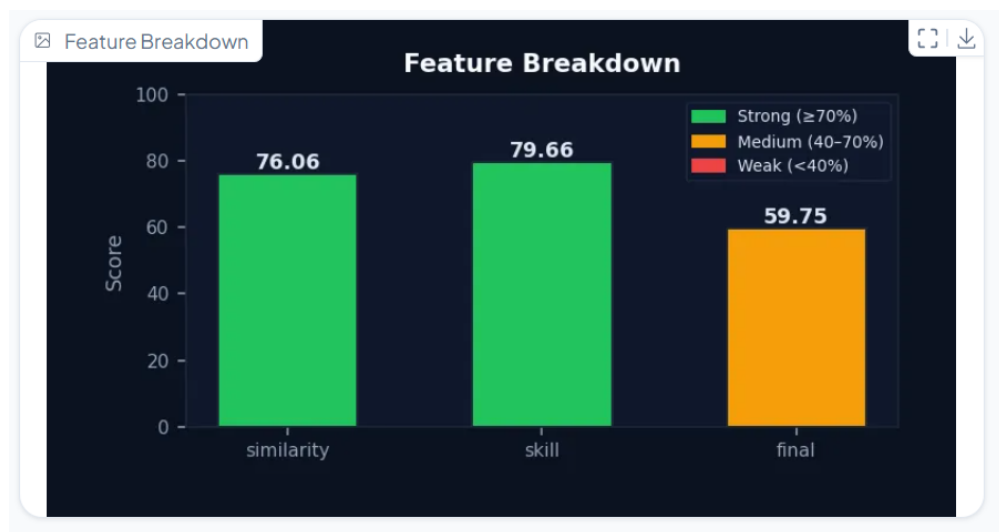


Figure 9.9: Feature Contribution Analysis using Bar Chart

The system provides visual representations of the evaluation results. The gauge chart displays the final score and shortlist probability, helping users quickly understand their overall performance. The bar chart illustrates the contribution of different features such as semantic similarity, skill match, and keyword density, providing transparency in the scoring process.

9.8 Detailed Feature Scores

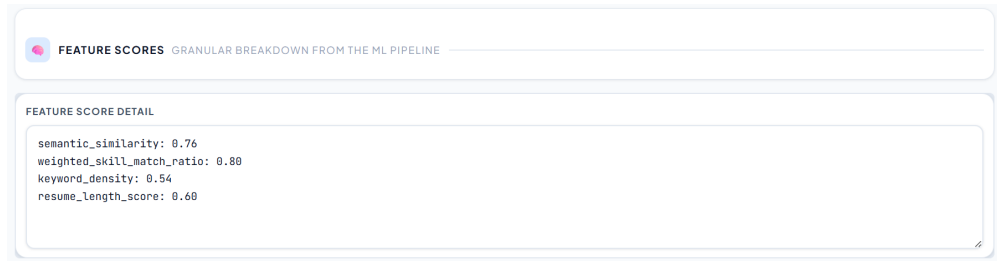


Figure 9.10: Detailed Feature Scores

This section displays numerical values of individual features such as semantic similarity, skill match ratio, keyword density, and resume length score used in computing the final score.

9.9 Skill Gap Analysis

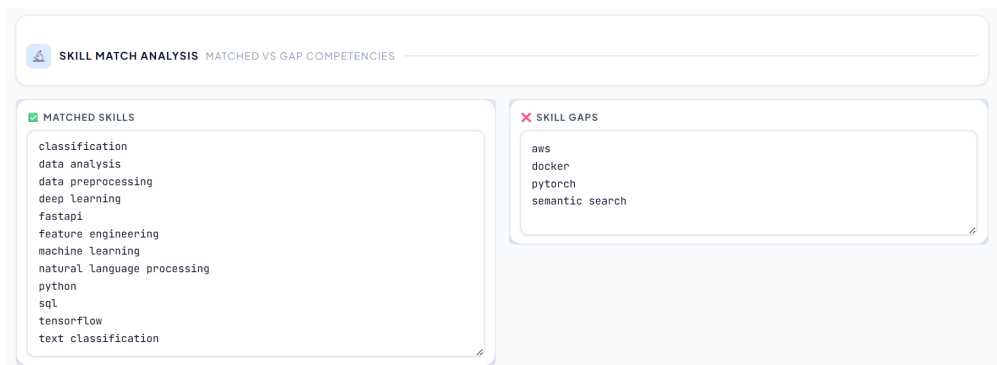


Figure 9.11: Matched Skills and Missing Skills

The system identifies matched skills and highlights missing skills required for the job role. This helps users understand gaps in their profile and areas that need improvement.

9.10 Analysis Explanation

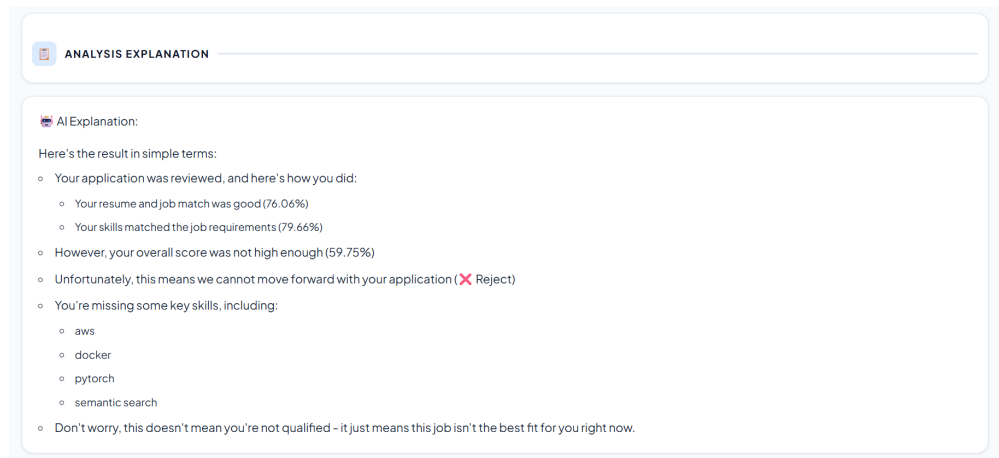


Figure 9.12: Decision Explanation Output

The system provides a clear explanation of the decision, including reasons for rejection or selection, eligibility evaluation, and missing core skills. This improves transparency and interpretability.

9.11 Learning Roadmap

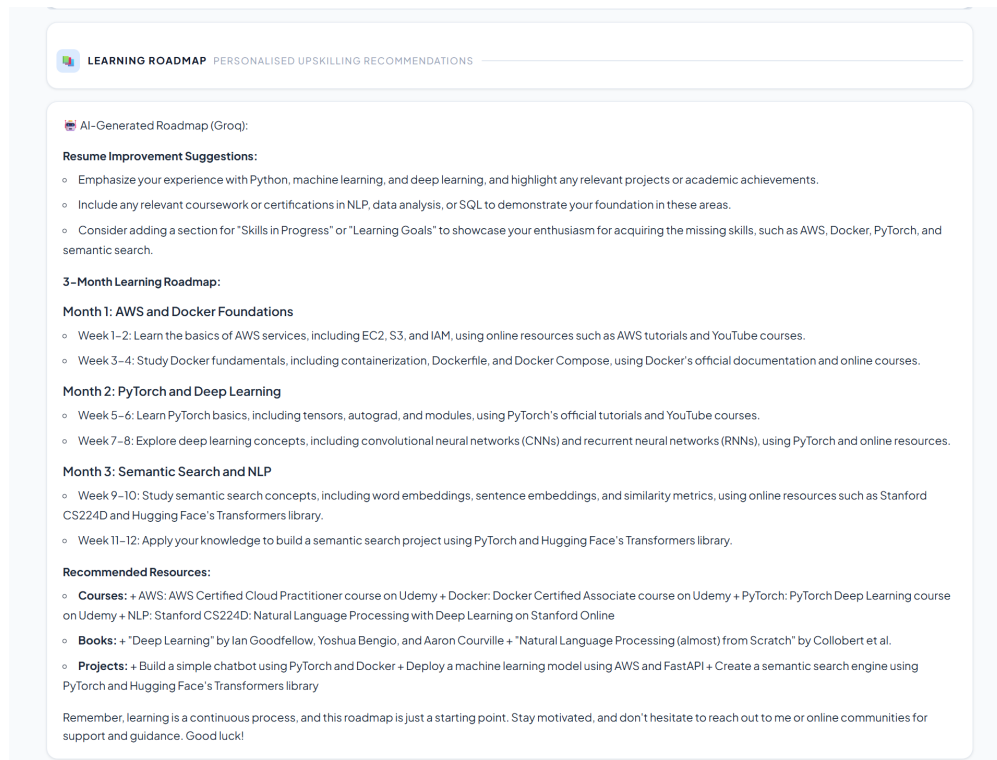


Figure 9.13: AI-Generated Learning Roadmap

A personalized learning roadmap is generated using LLM-based recommendations. It provides step-by-step guidance, suggested resources, and improvement strategies to enhance the candidate's skills and increase their chances of selection.

9.12 Case Analysis: Shortlisted vs Rejected

9.12.1 Shortlisted Case

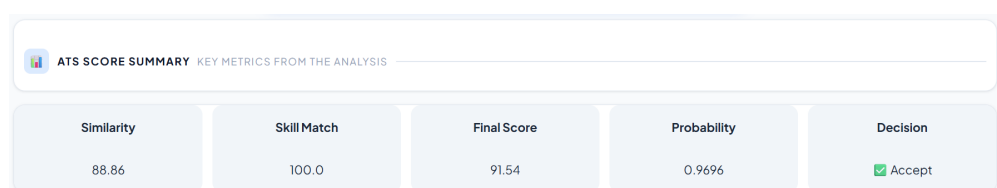


Figure 9.14: Example of Shortlisted Resume

In this case, the resume shows strong alignment with the job description. The system identifies high semantic similarity and strong skill matching. Most of the required and preferred skills are present, and eligibility criteria are satisfied. As a result, the final score exceeds the threshold, leading to a **shortlist decision**. This confirms that the system correctly identifies suitable candidates.

9.12.2 Rejected Case

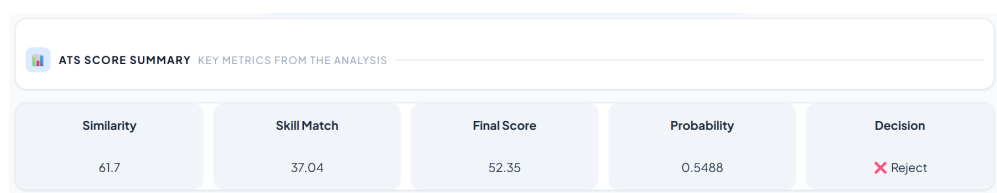


Figure 9.15: Example of Rejected Resume

In this case, the resume does not fully match the job requirements. Although some skills are present, important core skills are missing. This leads to a lower final score, which falls below the threshold. Hence, the system gives a **rejected decision**. This shows that the system can correctly filter out less suitable candidates.

Chapter 10

Conclusion and Future Scope

10.1 Conclusion

The proposed system, **CareerAlign AI**, presents an intelligent and structured approach to automate resume screening and improve transparency in the recruitment process. Traditional Applicant Tracking Systems (ATS) primarily rely on keyword matching or manual filtering, which often leads to inaccurate results and lack of clarity for candidates. The developed system addresses these limitations by integrating Natural Language Processing (NLP), semantic similarity, and rule-based scoring techniques into a unified framework.

The system accepts three primary inputs: resume, job description, and optional eligibility criteria. It performs text extraction and preprocessing using NLP techniques such as tokenization and lemmatization. The processed text is then converted into vector embeddings using a transformer-based model, and cosine similarity is computed to measure semantic alignment between the resume and job description.

In addition to semantic analysis, the system performs skill extraction and matching using a predefined skill taxonomy and alias mapping. A weighted skill matching mechanism ensures that both exact and semantically similar skills are considered. Eligibility constraints such as degree, CGPA, and experience are also validated to enhance the decision-making process.

A structured scoring model is applied by combining multiple features in-

cluding semantic similarity, skill match ratio, keyword density, and resume length score. The final score is transformed into a shortlist probability using a logistic function, providing an interpretable measure of selection likelihood. Based on a predefined threshold, the system classifies candidates into shortlisted or rejected categories.

Furthermore, the system improves user experience by providing explainable outputs, highlighting missing skills through skill gap analysis, and generating a personalized learning roadmap using LLM-based recommendations. This not only evaluates candidates but also guides them toward improving their profiles.

Overall, the proposed system successfully demonstrates an accurate, transparent, and scalable solution for resume evaluation. It bridges the gap between traditional ATS systems and intelligent recruitment platforms by combining rule-based logic with modern NLP techniques.

10.2 Future Scope

Although the proposed system performs effectively, there are several opportunities for further enhancement and extension:

- **Integration of Supervised Machine Learning Models:** The current system uses rule-based scoring. In future, labeled datasets can be generated using system outputs to train supervised models such as Logistic Regression, Random Forest, or Neural Networks to improve prediction accuracy.
- **Dynamic Skill Ontology Expansion:** The skill taxonomy can be expanded dynamically using real-world job data to include emerging technologies and domain-specific skills.
- **Advanced Semantic Understanding:** More powerful transformer models such as BERT, RoBERTa, or domain-specific LLMs can be

integrated to improve contextual understanding and semantic matching accuracy.

- **Real-time Industry Dataset Integration:** Integration with live job portals and datasets can help make the system more adaptive and aligned with current industry requirements.
- **Explainable AI Enhancements:** More detailed explanations using Explainable AI (XAI) techniques can be incorporated to provide deeper insights into scoring decisions.
- **User Profile Tracking and Personalization:** The system can be extended to track user progress over time and provide personalized recommendations based on historical performance.
- **Web and Mobile Application Deployment:** The system can be deployed as a full-scale web or mobile application to improve accessibility and usability for a wider audience.
- **Multi-language Support:** Support for multiple languages can be added to handle resumes and job descriptions in different languages.
- **Recruiter Dashboard Integration:** A recruiter-side dashboard can be developed to manage multiple candidates, visualize analytics, and make data-driven hiring decisions.

In conclusion, the system provides a strong foundation for intelligent resume screening and can be further enhanced into a fully automated, AI-driven recruitment assistant.

Bibliography

- [1] Pradeepa R, A C Charumathi, R Siva, S I Unais Sulthan, and B Vishnukumar. Intelligent resume evaluation tool based on machine learning for analysis and career advancement. *International Conference on Emerging Research in Computational Science (ICERCS)*, 2024.
- [2] Amit Pimpalkar, Aastha Lalwani, Roshan Chaudhari, Mohd Inshall, Mahak Dalwani, and Tarandeep Saluja. Job applications selection and identification: Study of resumes with natural language processing and machine learning. *IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, 2023.
- [3] Dhanalakshmi R V, Sushmitha K, Shivashish Gour, K J Nithin, Shashank M, and Keerthana S N. Ai-powered resume screening system using nlp and machine learning. *International Conference on Inventive Computing and Informatics (ICICI)*, 2025.
- [4] Tanjila Tamboli and Shakila Siddavatam. Ai-based resume shortlisting system using nlp and machine learning. *International Journal for Multidisciplinary Research (IJFMR)*, 8(1), 2026.
- [5] Tumula Mani Harsha, Mannuru Naga Rajeswari Pravallika, Gangaraju Sai Moukthika, Satish Anamalamudi, Dudipalli Siva Sai, and MuraliKrishna Enduri. Automated resume screener using natural language processing (nlp). *Proceedings of the Sixth International Conference on Trends in Electronics and Informatics (ICOEI)*, pages 1772–1776, 2022.

- [6] P. Felix Antony, P. Eshanth, and S. Julie Sharine. An intelligent resume–job matching system: Bridging candidates and opportunities through advanced semantic understanding. *2026 9th International Conference on Computational Intelligence in Data Science (ICCIDS)*, 2026.
- [7] Ghanashyam Vagale, P. Padma Priya Dharishini, Shashidhar Y Bhat, and Prajwal GK. Prospectcv: Llm-based advanced cv-jd evaluation platform. *2024 IEEE Students Conference on Engineering and Systems (SCES)*, 2024.